BillionBiz

The All-in-One Commerce Platform · Shoum Tech

Appointments & Bookings Guide

Everything you need to run a booking business on BillionBiz — explained in plain language. How to set up your services, your team and their working hours, how customers book you, how deposits and balances actually work, what every status and button does, and how to test the whole module before you go live — screen by screen and flow by flow.

Version 1.0 · September 2026

For Merchants, Store Teams & Release Testing

Covers first-run setup & presets · services, pricing, deposits and intake forms · team members, resources and working hours · the calendar and the day-money bar · the booking detail panel and every action on it · the customer booking journey · the life of a booking and its rules · deposits, balances and recording payments · notifications, reminders and timezones · plans, limits and staff

permissions · a full end-to-end test checklist with expected results · current limitations · troubleshooting · screen index · nine flowcharts covering every flow · glossary.

Table of Contents

Quick Start — Take Your First Booking in 10 Minutes

Part 1 — What Appointments on BillionBiz Looks Like

Part 2 — Getting Set Up — The First-Run Checklist

Part 3 — Your Services — What People Can Book

Part 4 — Your Team and Resources — Who and What Gets Booked

Part 5 — The Calendar — Running Your Day

Part 6 — The Booking Detail Panel — Every Action Explained

Part 7 — What Your Customer Sees — The Booking Journey

Part 8 — The Life of a Booking — Statuses, Holds and Rules

Part 9 — Money — Deposits, Balances and Recording Payments

Part 10 — Notifications, Reminders and Timezones

Part 11 — Plans, Limits and Staff Permissions

Part 12 — Testing the Module End to End

Part 13 — What Isn't Finished Yet

Part 14 — When Things Don't Look Right

Part 15 — Quick Reference — Screen Index

Appendix A — Release Notes, Test Environment and Verification Coverage

Part 16 — The Whole Module in Diagrams

Appendix B — What We Are Fixing, and What Turned Out Not to Be Broken

Glossary

Quick Start — Take Your First Booking in 10 Minutes

If you only read one part of this guide, read this one. Six steps, in this exact order, get you from an empty store to a real bookable appointment. The order matters: each step depends on the one before it, and the single most common support question about Appointments ("why are no times showing?") is almost always a step done out of order or skipped.

Step 1 — Check you can see the module. Left-hand sidebar → **Appointments**. If you land on the calendar, you're on a plan that includes Appointments. If you see an upgrade message instead, your subscription doesn't include it yet — see **Part 11 — Plans, Limits and Staff Permissions**.

Step 2 — Pick your booking rules (1 minute). Appointments → Settings. At the top is a *Getting set up* checklist, and below it **Start from a preset** — a row of business-type chips (salon, clinic, studio, and so on), each one carrying sensible defaults for minimum notice, how far ahead people can book, slot length, your cancellation window, no-show fee, reminder timing and default deposit. Click the one closest to your business and confirm. **Do this before you hand-tune anything**: applying a preset overwrites every booking rule with that preset's numbers, and it cannot be undone.

Step 3 — Add the person who will be booked (2 minutes). Appointments → Team & Resources → Add. Create a **Person** entry for yourself or a staff member. Rooms, chairs and equipment go here too, as non-person resources.

Step 4 — Give that person working hours (2 minutes). Still in **Team & Resources**, open the entry you just made and set at least one **Weekly shift** (for example Mon–Fri, 10:00–18:00). **This step is not optional.** A person with no shift is treated as never available, so no time slot will ever be offered for them — anywhere, to anyone — even though the person and the service both exist.

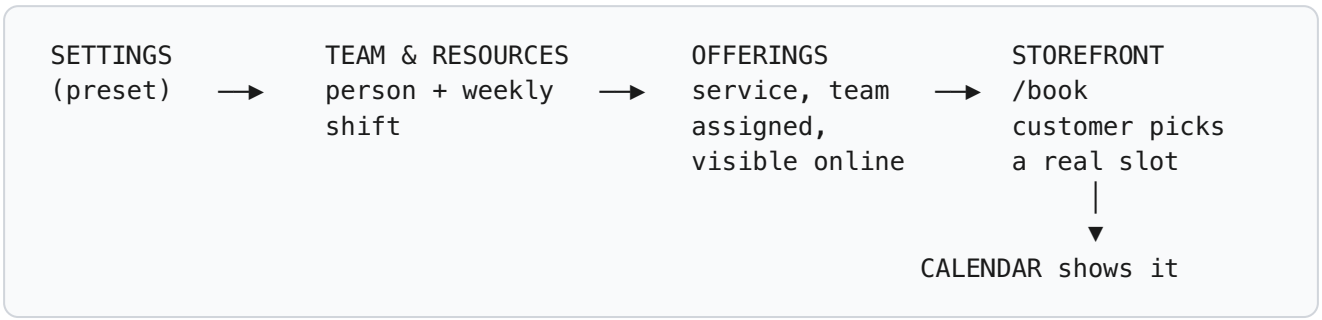
Step 5 — Create the service people will book (3 minutes). Appointments → Offerings → New offering. Fill in the name, the price, and how long it takes. Then, before you save:

- Under **Team**, tick the person from Step 3. An offering with nobody assigned offers no time slots on your storefront.
- Switch **Visible for online booking** on, or the service will exist for staff-booking only and customers browsing your store will never see it.

- If you want money upfront, set **Deposit type** and **Deposit value** (for example *Percentage of total* / `30`).

Save. If you want to ask customers questions when they book (allergies, vehicle registration, shirt size), reopen the offering and use **Manage intake questions** — that button only becomes available once the offering has been saved at least once.

Step 6 — Book it yourself, as a customer (2 minutes). Open your own storefront and add `/book` to the address — e.g. `yourstore.billionbiz.in/book`. Your service should be listed. Pick it, pick the person, pick a time, fill in the details, and confirm. Then go back to **Appointments → Calendar** and open the day you booked: the appointment is there, and clicking it opens the booking detail panel with every action available to you.



If no times appear at Step 6, don't change your prices or your rules — work through **Part 14 — When Things Don't Look Right**. In practice it is one of exactly three things: the person has no weekly shift, the offering has nobody assigned under **Team**, or the service isn't visible for online booking.

Part 1 — What Appointments on BillionBiz Looks Like

Appointments turns your BillionBiz store into a bookable-services business — hair salons, clinics, studios, repair shops, tutors, anything where a customer reserves a specific person, room, or piece of equipment for a specific block of time. This part covers everything you, the merchant (or your staff), see and control from your own dashboard: setting up the services people can book, building your team's schedule, running the calendar day to day, and collecting money for bookings.

Finding Appointments

Nav path: Left-hand sidebar → **Appointments**.

If your plan doesn't include Appointments yet, the page shows an upgrade message instead of the calendar — see **Part 11 — Plans, Limits and Staff Permissions**.

Once you're in, four tabs run along the top of every Appointments screen:

```
[ Calendar ]  [ Offerings ]  [ Team & Resources ]  [ Settings ]
```

- **Calendar** — the day-to-day booking board (`/appointments`)
- **Offerings** — the services people can book (`/appointments/offerings`)
- **Team & Resources** — your staff, rooms, and equipment (`/appointments/resources`)
- **Settings** — booking rules, reminders, and the first-run checklist (`/appointments/settings`)

On a narrow screen or a squeezed sidebar layout, these four tabs collapse into a single dropdown labelled "Section" so the active tab's name stays readable instead of getting clipped.

One important note on who sees what. If you are logged in as a team member who is also linked to a bookable resource (see Part 4), you land on a simplified "My Day" view instead of the full calendar, and you only see **Calendar** and a personal **My availability** screen — Offerings, Team & Resources, and Settings are admin/owner-only and won't appear for you.

Part 2 — Getting Set Up — The First-Run Checklist

The first time you open Appointments, before you've added any services or team members, the Settings tab shows a short getting-set-up checklist:

- **Add your first service** — ticks itself off automatically once you've created at least one Offering.
- **Add a team member or resource** — ticks off once you've added at least one entry in Team & Resources.
- **Set booking rules** — ticks off once you've set a default deposit percentage/amount or a no-show fee percentage in Settings.
- **Set reminder cadence** — ticks off once at least one reminder time is configured in Settings.

Each item with a clickable label jumps you straight to the right tab. The checkmarks are never faked — they only turn green once the real underlying thing exists (a real service, a real team member, a real saved setting), so if a box is unticked, that thing genuinely hasn't been done yet.

Start from a preset

Right below the checklist is **Start from a preset** — a row of chips, one per business type BillionBiz ships default booking rules for (minimum notice, booking horizon, slot length, cancellation window, no-show fee, reminder timing, deposit default). Clicking a preset opens a confirmation dialog:

> **Apply this preset?** Applying "*[preset name]*" replaces every booking rule below with this preset's defaults, even ones you've already saved. This can't be undone.

Confirming overwrites **all** of your currently saved booking rules with that preset's numbers — it is not a merge, and there is no undo. If you've already tuned your settings by hand, applying a preset again will wipe those changes. Presets are a starting point, meant to be used once, early — not something to reach for casually later.

Settings screen — states, validation, and what to expect

- **First load / empty state:** before you've saved anything, the six booking-rule fields simply show whatever defaults the platform supplied — there's no separate "empty" placeholder screen. The checklist card (above) is effectively the first-run indicator for this screen.
- **Loading state:** while offerings, resources, settings, and presets are all still being fetched, the whole tab shows a skeleton rather than a half-filled form.
- **Error state:** if the initial settings/offerings/resources/presets load fails, the tab shows a dedicated load-error message with a retry action instead of a broken form.
- **Read-only / permission state:** the **Save** button is disabled only while a save is already in flight (`saving`); there is no separate role-based read-only rendering of this form confirmed in source — anyone who can reach the Settings tab at all can edit and save it.
- **Validation — finding:** the six numeric booking-rule fields (Minimum notice, Booking horizon, Slot length/start increment, Cancellation window, No-show fee %, Default deposit) have **no validation of any kind** before save. There is no minimum, no maximum, no negative-number check, and no error styling wired to any of these fields — the Save action simply reads each field as a number and saves it. Concretely: a merchant can type `-30` into Minimum notice, or `0` into Cancellation window, or `500` into No-show fee %, click **Save**, and it will save with no warning message of any kind. Expected result for a tester: entering a negative or absurd value in any of the six numeric fields and clicking Save **succeeds silently** — no validation message appears, and the value persists on reload. This is a genuine gap, not an oversight in this audit.
- **Reminder cadence field** takes comma-separated numbers (minutes-before) as free text with a helper hint ("e.g. 1440,60 sends reminders a day before and an hour before") — anything that isn't a number is silently dropped on save rather than rejected with an error.
- **Prerequisite:** none — Settings has no dependency on offerings or team members existing first; it can be edited before anything else in the module is set up.

Part 3 — Your Services — What People Can Book

Nav path: Appointments → **Offerings**.

This is the master list of every bookable service your store offers — a haircut, a consultation, a repair slot, a class. Each row shows **Name**, **Price**, **Duration**, and **Status** (Active/Inactive), with **Edit** and **Deactivate** actions. A search box filters by name; an **Active only** toggle hides deactivated services from the list.

Deactivating an offering (rather than deleting it) removes it from booking without destroying its history — the confirmation reads: *"[name] will no longer be bookable. This can be reversed by re-activating it later."* There is no delete option; only activate/deactivate.

If you haven't added any offerings yet, the list shows an empty-state message — *"Offerings are the bookable services people can schedule — create one to get started"* — with a button to add your first one.

Offerings list — states and expected results

- **Loading:** a table-shaped skeleton while the list fetches, shown only on the very first load (not on every refetch — an already-populated list doesn't flash a skeleton on refresh).
- **Error:** if the list fails to load, the table area is replaced with an error message (the raw error text) instead of the skeleton or rows — there is no dedicated retry button on this specific error block (unlike the New Appointment page's load-error, which does have one).
- **Empty:** distinct from error — a genuine empty state only shows when the list legitimately has zero offerings and no search term is active.
- **Deactivate — expected result:** clicking Deactivate and confirming removes the offering from the Active list immediately and it stops being bookable on the storefront; toggling **Active only** off still shows it, tagged Inactive, and it can be re-activated (see gap note below on where that control lives).

Creating or editing an offering

Clicking **Add offering** (or **Edit** on an existing row) opens the full offering form, organised into labelled sections:

Basics | Field | What it does | |—|—| | Name | What customers see when browsing/booking this service. Required. | | Category | Optional grouping label. | | Description | Optional longer text shown to customers. | | Image | Optional photo for the service. |

Duration & buffers — *"How long the appointment takes, plus any prep or cleanup time blocked around it."* | Field | What it does | |—|—| | Duration (mins) | The length of the appointment itself — what the customer books. | | Processing time (mins) | Extra time blocked on the resource's schedule after the appointment (e.g. equipment sterilising, a room resetting) that the customer doesn't pay for but the calendar keeps clear. | | Buffer time (mins) | Additional gap time blocked around the booking, distinct from processing time. |

Pricing & deposit — *"What people pay, and whether a deposit is collected upfront."* | Field | What it does | |—|—| | Price type | **Fixed** (an exact price) or **Starting at** (a from-price, e.g. for services that vary). | | Price | The amount, in your store's currency. | | GST rate (%) | Must be between 0 and 100; used for tax calculation on the booking. | | SAC code | Optional service-tax classification code. | | Deposit type | **No deposit**, **Percentage of total**, **Fixed amount**, or **Full prepayment**. | | Deposit value | The percentage or fixed amount, shown only when a deposit type other than "No deposit" is picked. |

The **deposit type** you choose here is what a customer is asked to pay upfront when booking online, and it's also what drives the pre-filled amount admins see later when sending a payment link or recording a payment against a booking of this service (see Part 9).

Booking rules & visibility — *"How many people can book at once, and whether this shows up for online booking."* | Field | What it does | |—|—| | Booking model | **One person at a time**, or **Group / capacity** (multiple customers can book the same time slot up to a limit). | | Capacity | Only shown/required when Booking model is set to Group/capacity — the maximum number of people per slot. | | Visible for online booking | A switch controlling whether customers can find and book this service themselves online, versus it only being bookable by staff from inside the admin panel. |

Team — *"Who can be booked for this offering."* A multi-select list of your Team & Resources entries (people only) who can perform this service. Leaving it empty means the service has no assigned staff yet — see the gap note in Part 13.

Intake form — *"Collect extra details from people before their appointment."* This section doesn't hold the actual questions — it shows how many custom questions are configured (or "No custom questions yet") and a **Manage intake questions** button that opens a separate screen (see below). That button is disabled with the tooltip *"Save this offering first to add intake questions"* until the offering has been saved at least once — you cannot build the intake form on a brand-new, unsaved offering.

Saving the form validates required fields, and if your plan has a cap on the number of offerings you can create, saving a new one beyond that cap shows an inline plan-limit message rather than a popup toast — see Part 11.

Offering form — validation, edge cases, and expected results

Field	Validation on Save	Message shown	Finding
Name	Must not be blank	“Name is required”	—
Price	Rejects blank and negative	“Enter a valid price”	**Gap: a price of exactly `0` passes this check and saves successfully** — the check is “blank OR negative,” not “greater than zero,” so a free/zero-priced service can be saved with no warning at all.
Duration (mins)	Must be blank-or-zero rejected — i.e. must be a number greater than 0	“Enter a valid duration”	Correctly rejects `0` and negative values (unlike Price).
Capacity	Required and must be `> 0`, but **only enforced when Booking model = Group/capacity** — not checked at all for one-person-at-a-time offerings	“Enter a capacity greater than 0”	—
GST rate	Must be empty or a number from 0–100 inclusive	“Enter a GST rate from 0 to 100”	—
Deposit value	For Percentage or Fixed amount deposit types: must be non-blank, a finite number ≥ 0; Percentage mode additionally caps it at 100	“Enter a valid deposit value (0-100 for percentage, 0 or more for a fixed amount)”	Not checked at all when Deposit type = “No deposit” (field isn’t shown).

No duplicate-name check exists — two offerings with the identical name save without any warning; nothing in the form or the list prevents or flags this.

Expected results for a tester: - Leaving Name blank and clicking Save → form stays open, "Name is required" appears under the field, nothing is submitted. - Entering price `0` and clicking Save → **saves successfully with no error** (this is the finding above, not a bug to "fix" during testing — confirm it reproduces as described). - Entering duration `0` or a negative number and clicking Save → blocked, "Enter a valid duration" shown. - Setting Booking model to Group/capacity and leaving Capacity blank → blocked, "Enter a capacity greater than 0." - Entering GST rate `101` → blocked, "Enter a GST rate from 0 to 100." - Creating a second offering with the exact same name as an existing one → **saves successfully, no warning** (finding). - Saving beyond your plan's offering cap → form stays open, an inline banner with an upgrade link appears in place of a popup toast; no offering is created.

States: - **Loading (edit mode):** the form shows a form-shaped skeleton while the existing offering's data loads. - **Load error (edit mode):** a toast error appears ("Couldn't load…"-style message) if the existing offering fails to load; the form is left effectively unusable underneath. - **Plan-gated:** covered above — an inline `Alert` banner replaces the normal submit flow, not a blocking screen. - **Prerequisite:** none to open/save the basic form — but the **Manage intake questions** button on this form is itself gated: it stays disabled until the offering has been saved at least once (tooltip: *"Save this offering first to add intake questions"*). Trying to click it on a brand-new unsaved offering does nothing but show that tooltip.

Building the intake (booking) form for a service

From an already-saved offering's **Manage intake questions** button, you reach a dedicated question builder:

- **Add a question** opens a dialog where you set: the question text ("Question"), the answer type ("Answer type" — see the list below), whether it's required, optional placeholder text, optional help text, and — for dropdown/multiple-choice types — a list of choices, one per line.
- Answer types available: Short answer, Long answer, Number, Dropdown (single choice), Multiple choice, Checkbox, Date, Time, Date & time, Email, Phone number.
- Questions can be **reordered by dragging** (a drag handle on the left of each row), **edited**, or **removed**.
- A live **preview pane** on the right renders exactly what the customer will see and answer, in the order you've set — including a disabled "Submit" button (it's a preview, not a real form).
- Changing questions and navigating away without saving triggers an unsaved-changes warning dialog.

- A separate **Submissions** panel on this screen is still a stub — it shows an informational "coming soon" note rather than a roll-up list of everyone's answers. This is a convenience gap, not a data gap: every answer a customer submitted is readable on that booking's own detail panel (Part 3), correctly formatted for each question type. A roll-up view here is on the fix list — see **Appendix B**.

Validation on this screen: the Add-question dialog requires a non-blank question text ("Enter a question") and, for choice-type questions (Dropdown/Multiple choice), at least one option ("Add at least one choice"). Expected result: leaving the question text blank and trying to save the question → blocked with "Enter a question"; adding a Dropdown question with zero choices listed → blocked with "Add at least one choice." No duplicate-question-text check exists. **Prerequisite:** this whole screen is unreachable until the parent offering has been saved once (see above).

Part 4 — Your Team and Resources — Who and What Gets Booked

Nav path: Appointments → **Team & Resources**.

This list holds everything a customer's appointment can be booked *against* — staff, plus optionally rooms or equipment. Each row shows **Name**, **Type** (Person / Space / Equipment), **Linked user** (whether the resource is tied to a real staff login, or "—" if not), **Status** (Active/Inactive), and actions: an **Availability** icon-link (always available), and **Edit** / **Deactivate** (only if you have permission to manage the team).

A **type filter** (All types / Person / Space / Equipment) and a search box narrow the list. If you haven't added anyone yet, an empty state prompts you to add your first team member.

Staff-seat usage banner

If your plan limits how many *person*-type resources ("staff seats") you can have active, a banner sits above the list showing one of:

- *"[used] of [limit] staff seats used"* — you're within your plan's limit.
- *"[used] of [limit] staff seats used — you've reached your plan's staff limit"* — you're at the cap; adding another person requires buying an extra seat.
- *"[used] staff added, but your plan doesn't include any staff seats — buy a seat to keep your team active"* — most plan tiers start with **zero** staff seats included by default; staff seats are an add-on purchase rather than something every plan tier automatically unlocks. A **Buy seat** button on this banner links to the billing add-ons screen.

Space and equipment-type resources are not affected by this staff-seat limit — it applies to Person-type entries only.

Team & Resources list — states and expected results

- **Loading:** table skeleton on first load only.
- **Error:** the table area is replaced with a plain error message (no dedicated retry button on this block).
- **Empty:** a distinct empty-state prompt, separate from the error state, shown only when there are genuinely zero resources and no search/filter active.
- **Deactivate — expected result:** confirming removes the resource from being offered for new bookings immediately; the person's staff login to the rest of the admin panel is untouched.

Adding or editing a team member / resource

- **Type**: Person, Space, or Equipment. Locked (can't be changed) once you're editing an existing entry.
- For **Person** type, you choose one of two modes:
  - **Link an existing user instead of inviting a new one** — toggle on, then enter the existing user's ID to attach the resource to an already-existing staff login.
  - Toggle off (default): fill in **Name**, **Email**, and a password — this creates a brand-new staff account at the same time as the resource.
- **Skill tags** — free-text comma-separated tags (e.g. "haircut, colouring") for your own organisation; not currently used to filter bookings.
- **Bookable online** — a switch controlling whether this resource can be selected by customers booking online versus staff-only booking.
- **Set an initial weekly shift** (create mode only) — an optional one-time shortcut: pick a day of the week plus a start and end time to seed the very first availability rule for this resource. The form explicitly warns: *"Optional — skip this and add shifts later from Manage availability. Without one, this resource shows as never-available until a shift is added."* This matters — a newly created staff member or resource with no shift set up cannot be booked at all until someone adds availability.

Deactivating a resource shows a confirmation — *"[name] will no longer be bookable. This does not affect their user account"* — meaning deactivating the appointments-side resource does **not** disable the person's staff login to the rest of the admin panel.

Add/edit dialog — validation, edge cases, and expected results

Field / situation	Validation	Message	Finding
Name	Must not be blank	“Name is required”	No duplicate-name check — two resources with the identical name save without warning.
Link-existing mode, create only	Existing user ID must not be blank	“Enter the existing user’s ID”	No format/existence check confirmed beyond non-blank in this dialog — whether an invalid ID is rejected happens server-side, not shown as a client validation message here.
Create-new-person mode, create only	Email must not be blank	“Email is required”	**No email-format validation** — any non-blank string passes this client-side check (e.g. “abc” would pass here; format checking, if any, happens only server-side).
Create-new-person mode, create only	Password must not be blank	**Hardcoded English string “Password is required”**	**i18n inconsistency finding** — every other validation message on this dialog runs through the translation system; this one is a raw English literal that would not translate for non-English merchants.
“Set an initial weekly shift” (create only), if toggled on	Shift end time must be strictly after start time (equal times also fail)	“End time must be after start time”	—

Expected results for a tester: - Leaving Name blank → blocked, “Name is required.” - Creating a resource with the exact same name as an existing one → **saves successfully, no warning** (finding). - Create-new-person mode, leaving password blank → blocked with the untranslated “Password is required” string regardless of the admin panel’s active language. - Toggling “Set an initial weekly shift” and setting start = end (e.g. both 09:00) → blocked, “End time must be after start time.” - **Prerequisite / failure symptom:** a resource saved with no shift at all (either skipped the optional seed-shift, or it’s an edit where none was ever added) shows as never-available — it will not offer any time slot to a customer or to staff booking on their behalf, and no error is shown at booking time explaining why; the only way to discover the cause is to open that resource’s Availability screen and see it’s empty. This is explicitly called out on the dialog itself.

Setting working hours (Availability)

Clicking the **Availability** icon-link on any resource’s row opens that resource’s schedule editor:

Weekly shifts — recurring working hours. Each shift is a day of the week plus a start and end time (e.g. Monday, 09:00–17:00). Add as many shifts per resource as needed (e.g. split morning/afternoon shifts) or remove them. If a resource has none, the screen reminds you it will show as never-available until you add one.

Date overrides & blocks — one-off exceptions to the weekly pattern, each with a title and a start/end date-and-time. This is where you block out time off, holidays, or a single day’s different hours. There is no separate calendar-side “block this slot” button — blocking a resource’s time is done entirely from this screen, never from the day-to-day Calendar view itself.

This same editor screen is reused, unchanged, when a staff member manages their **own** availability (see below) — it’s the identical control, just reached from a different starting point and scoped to one resource automatically.

Availability editor — validation and a significant gap

- **Weekly shift validation:** end time must be after start time — “End must be after start.” No other check.
- **Date override/block validation:** both a start and end date-time are required (“Choose a start and end”); if both are set, end must be after start (“End must be after start” — the same message as the shift check).

- **Finding — no overlap detection anywhere in this editor.** You can add two weekly shifts for the same resource that cover the same day and overlapping hours (e.g. Monday 09:00–17:00 and Monday 12:00–20:00), or two date-override/blocks that overlap in time, and both save with no warning. The system does not merge, reject, or flag the overlap — it is left to the merchant to notice. Expected result for a tester: adding a second, time-overlapping weekly shift or block for the same resource and saving both → **both save silently, no validation message appears.**
- **Prerequisite / failure symptom:** as noted above, zero shifts = the resource shows as never-available; there is no explicit on-screen warning about this at booking time elsewhere in the product, only the reminder text on this editor screen itself and on the add-resource dialog.

If you're a staff member: My availability

Staff who are linked to their own resource get a personal **My availability** screen (reached from their own nav, not from Team & Resources — they don't see that admin list at all) with the same weekly-shifts / date-overrides-and-blocks editor, scoped to themselves. If a staff login has no linked resource yet, the screen shows: *"Your account isn't linked to a team-member resource yet — ask an admin to link it in Team & Resources."*

Part 5 — The Calendar — Running Your Day

Nav path: Appointments → **Calendar** (this is also the very first thing you land on when you open Appointments).

Day, week, and month views

Three view buttons switch how much of your schedule you see at once:

- **Day** — one resource per horizontal lane, hour-by-hour, the most detailed view. This is where you spend most of your time.
- **Week** — one column per day, condensed booking chips.
- **Month** — a traditional month grid; clicking any day jumps you straight into that day's Day view.

Use the ← / → arrows (or **Today**) to move through dates, and on Day/Week view a **Jump to now** button snaps the view back to the current time.

A **source filter** dropdown narrows the view to bookings from a particular channel (online, booked by staff, booked in person, etc.) or shows all.

The day-money summary bar

At the top of the Day view sits a summary strip showing, for the selected day:

- **Collected** — the total amount actually taken in so far today.
- **Due at counter** — what's still expected to be collected today for today's bookings.
- **[count] booked today**, **unsold time** (how much bookable time on your team's schedule went unfilled), and **no-show** count (with the total no-show fee amount if any no-shows have a fee).
- **Unassigned bookings** — a count of bookings not yet assigned to a specific team member.
- **Deposit owed** — appears only on days where a deposit was actually expected; it's a breakdown of the "due at counter" figure, not an amount on top of it.

These numbers are all pulled from real payment records for the day — none of them are estimates or placeholders. If the figures haven't loaded yet, the bar shows a loading dash (`–`) rather than a stale or fabricated number, and if the figures fail to load, every cell instead shows the word "unavailable" rather than a zero or a guess — a zero on this bar always means genuinely zero, never "couldn't load."

Reading the day grid

Each row is a resource (staff member, room, or equipment); each colored block is a booking, positioned and sized by its actual start time and duration. Hovering a booking shows the customer's name and its current status. For services with multiple back-to-back steps against the same resource, a small sub-rail under the booking chip shows alternating busy/free segments so you can see real idle gaps between steps at a glance.

Booking capacity note: for "Group / capacity" services, the system only offers a time slot to a customer (or to you, when booking on their behalf) if room remains under that offering's capacity — but the calendar day-grid does not display a running "X of Y spots left" number on the booking itself. If you need to know how full a group slot is, you'll need to open the booking(s) individually.

- **Click any existing booking** to open its detail panel (Part 6).
- **Double-click an empty spot** in any resource's lane to open the New appointment screen, pre-filled with that resource and roughly that time.
- Two floating buttons in the bottom corner (a walk-in shortcut and a "New booking" button) also open the same New appointment flow from anywhere in the calendar.

If a day/week/month has nothing booked, an empty-state message appears with its own "New appointment" button.

Creating a new appointment

Two ways to reach the booking form: the quick popover (from the calendar's floating buttons or a lane double-click) or the full page at **Appointments → New appointment** (reached via the popover's "full form" link). Both share the same fields:

1. **Add contact details** switch — turn this on for a genuine walk-in (records the booking as "booked in person" rather than "booked by staff").
2. **Use an existing contact** switch — search your existing customers by name/phone/email and select one, or turn it off to type a new contact's **Contact name**, **Contact phone**, and optional **Contact email** directly (no separate customer record needs to exist beforehand).
3. **Date**.
4. **Service** — pick from your Offerings, shown with its duration.
5. **Team member** — pick a specific person, or leave it on **Anyone available** to let the system assign whoever has room.
6. **Time** — a row of clickable time buttons for the chosen service/team member/date; if nothing is available, a "No open slots for this selection" message appears instead.
7. **Internal notes** — free text visible only to your team.

Submitting creates the appointment and returns you to the calendar. If the same slot gets taken by someone else in the moment between you loading the page and submitting, the booking is rejected and you're shown an error rather than silently double-booking the resource.

New appointment — validation, states, and expected results

Situation	Message
No service chosen, tries to submit	"Choose a service"
No time slot chosen, tries to submit	"Choose a time"
Walk-in (quick popover), no contact entered	"Enter a contact name and phone"
Full-page form, no existing/new contact resolved	"Choose an existing contact or enter a name and phone"
No open time slots for the chosen service/team-member/date combination	"No open slots for this selection." (informational text, not an error)

Prerequisites and failure symptoms: - If you haven't created any offerings yet, the Service dropdown is simply empty — there is no explicit "add a service first" message on this form itself; the symptom a merchant sees is an unselectable/empty picker. - If the chosen service has no team member assigned to it, or the only resource that could serve it has no working-hours shift configured, the Time-slot row renders the "No open slots for this selection" message — this is the single most likely support case (Part 13) and this exact message is the symptom to look for when diagnosing it. - **Loading state:** the full-page form shows its own loading indicator while offerings/resources first load; if that initial load fails, the whole form is replaced by a load-error block with a retry action ("Couldn't load the services and team members needed for this appointment."). - **Slot-fetch error:** if fetching available times for a specific service/date/resource combination fails (separately from the initial page load), a dedicated error block with its own retry button appears in place of the time-slot row ("Couldn't load available times for this selection.") — this is distinct from, and takes priority over, the "No open slots" empty-result message. - **Expected result:** submitting a fully valid form creates the appointment, returns you to the calendar, and the new booking appears immediately in the correct resource's lane at the chosen time with status "Awaiting payment" or "Confirmed" depending on whether a deposit is required.

Part 6 — The Booking Detail Panel — Every Action Explained

Clicking any booking, anywhere (calendar chip, My Day card, a deep link from a customer's own profile), opens the same detail panel — either as a column beside the calendar on a wide screen, or as a full overlay on a narrower one.

What's shown, top to bottom:

- **Identity header** — the customer's name/initials and a human-readable reference number for the booking (this reference is generated for display purposes only — it is not an official order or invoice number).
- **When & where** — the date, time, and status chip.
- **Contact** and **source** — how the booking was made: Booked online, Booked by staff, or Booked in person.
- **Checked in at** — only shown once the customer has actually been checked in.
- **Services** — each service line on this booking, with the assigned team member.
- **Intake form answers** — the customer's answers to that service's intake questions, if it has any. If the service has no intake form, this section doesn't render at all. If it has a form but no answers came back, you'll see: *"No intake answers were submitted for this booking."*
- **Activity** — a real timeline of what's actually happened to this booking (booked, checked in, started, completed, cancelled — with a reason if one was given, or marked as no-show). This list only ever shows events that truly occurred; there is no "deposit captured" or "reminder sent" entry, because those aren't tracked as logged events yet (see Part 13).
- **Payment** — covered in full in **Part 9 — Money**.

Status chips and what each one means

Status shown to you	What it means
Awaiting payment	Booking created but the customer hasn't completed payment yet.
Confirmed	Payment is settled (or no payment was required) and the slot is locked in.
Checked in	The customer has arrived.
In progress	The service is actively underway.
Completed	The appointment is done.
Cancelled	The booking was cancelled.
No-show	The customer never arrived.

Action buttons and what they do

The buttons offered change depending on the current status — you'll never be offered an action that isn't actually valid from where the booking currently stands:

From status	Buttons available	Effect
Awaiting payment	Cancel	Cancels the pending booking.
Confirmed	Check in, No-show, Reschedule, Cancel	Check in moves it to "Checked in." No-show marks it missed. Reschedule opens the reschedule dialog (only offered at this stage). Cancel cancels it.
Checked in	Start, Cancel	Start moves it to "In progress."
In progress	Complete, Cancel	Complete finishes the appointment.
Completed / Cancelled / No-show	*(none)*	These are end states — no further action buttons appear.

Cancel always opens a confirmation dialog first — *"Cancel this appointment? This releases the booked slot and notifies the customer. This cannot be undone."* — with an optional reason text field.

Reschedule opens a dialog with a new date, an optional different team member, and a fresh set of time slots for that combination — it deliberately still offers the booking's *own current* time slot as an option (rather than treating it as already taken by itself), so rescheduling to the exact same time a resource was otherwise fully booked still works.

Action-button expected results (for testers)

Action	Expected end state
Check in (from Confirmed)	Status chip changes to "Checked in"; a "Checked in at" timestamp appears in the panel; a matching Activity entry is added.
No-show (from Confirmed)	Status chip changes to "No-show"; if the offering/settings have a no-show fee configured, that fee amount reflects in the day-money bar's no-show total.
Start (from Checked in)	Status chip changes to "In progress."
Complete (from In progress)	Status chip changes to "Completed" — an end state; no further action buttons appear.
Cancel (from any cancellable status)	After confirming the dialog, status chip changes to "Cancelled," the slot frees up on the calendar for that resource/time, and the booking is an end state with no further actions.
Reschedule (from Confirmed only)	After picking a new date/time and confirming, the booking moves to the new slot, the old slot frees up, and the status remains "Confirmed."

Reschedule dialog — validation and error state

- **Validation:** a time slot must be chosen before confirming — "Pick a new time slot." No other client-side validation on this dialog.

- **Conflict on submit:** if the newly chosen slot gets taken by someone else between opening the dialog and confirming, the server refuses with a slot-already-taken conflict; the failure is surfaced as a toast error, and the dialog **stays open** rather than closing on failure — the merchant can pick a different slot and retry without starting over.
- **Expected result:** confirming a valid, still-open new slot closes the dialog, moves the booking, and the calendar reflects the new position immediately.

Part 7 — What Your Customer Sees — The Booking Journey

This part covers what a merchant's own customer sees and does when booking a service on your storefront — from finding your services to managing a booking afterwards. It also doubles as the release-testing script for this module: every screen lists its states (loading / empty / error) and the exact result a tester should see for each negative or edge path, so this document can be walked step-by-step against a live store.

Reaching the booking page

Your booking page lives at **yourstore.com/book**. It is always reachable — whatever your homepage is set to, whether you sell products, services, or both, and whether the visitor is signed in or browsing as a guest. There are two ways a customer lands on it:

1. **It is your homepage.** If your store is set up in "booking" mode, `/book` IS what customers see when they visit your site's main address.
2. **A direct link.** Any link or button that points to `/book` (for example, a "Book Now" button you place on a page, or a link you share) opens the booking page directly.

A signed-in customer who has booked before can also reach their **existing** bookings from **My Orders → Bookings** — but this tab only appears once your store has services turned on, and it shows past bookings, not a way to start a new one.

There is currently no button in the main site menu/header that takes a visitor straight to `/book` — a customer only gets there via your homepage (if set to booking mode) or a link you place somewhere on your site.

The service list

The booking page opens on a list of your services ("offerings"). For each one, the customer sees:

- A photo (if you've added one) or, if not, a simple monogram badge in your brand colour — never a blank grey box
- The service name and (if you've written one) a short description
- The duration (e.g. "45 min" or "1 hr 30 min")
- The price

If you've priced a service as "starting at" rather than a fixed price, the customer sees "Starting at ₹X" instead of a flat number. If the service requires a deposit, that is not shown as a separate number on this list — the customer only finds out a deposit is needed once they reach the review step near the end of the flow (see "Deposits and paying" below).

The service list itself can be laid out four different ways (a photo grid, a list of rows, a horizontal scrolling rail, or the original card-grid layout) — this is a look-and-feel choice set on your side, not something the customer controls. Whichever layout is used, the same information (photo, name, description, duration, price) is always shown.

Tapping a service takes the customer to the next step in the flow.

Screen states — expected results:

State	What the tester should see
Loading	Three skeleton placeholder bars where the service list will appear
Error (services failed to load)	"We couldn't load services right now. Please try again." — no service list shown
Empty (no services published)	"No bookable services are available right now." — no crash, no blank page
Normal	Service cards/rows as described above, "Select" affordance on each

No signed-in/guest difference on this screen — it looks and behaves identically either way.

Choosing a staff member or resource

If a service can be delivered by more than one person/resource (e.g. different stylists, rooms, or equipment), the customer next sees a list to choose from, plus an "Any available" option. Each entry shows a photo (or an icon if none is set) and a name.

This screen is skipped automatically, and the customer is taken straight to picking a time, when: - there is only one eligible staff member/resource for that service, or - you've configured the service to not use staff/resource selection at all, or - the service is a group/class-style ("capacity") service that isn't tied to one specific person

Screen states — expected results:

State	What the tester should see
Skipped	Tester lands directly on the date/time screen; no staff picker shown at all
Empty (nobody eligible)	“No one is available to book for this service right now.”
Error	“We couldn’t load available options right now. Please try again.”
Normal	List of staff/resources plus an “Any available” card

No signed-in/guest difference here either.

Picking a date and time

The customer sees a horizontal strip of the next 14 days to choose a date, and a list of available time slots for the selected day.

- **The moment the customer taps a time slot, that slot is held for them for 5 minutes** — nobody else can take it during that window. A countdown timer appears (both as a bar at the bottom of the screen on mobile and alongside the summary on desktop) showing the time left, e.g. “4:32 remaining.” In the final minute, the countdown turns red/bold to draw attention.
- **If the hold expires** before the customer finishes booking, they are bounced back to the time-slot screen with a message telling them their hold expired (“Your held slot expired. Please pick a new time.”), and they must pick a slot again.

One thing to flag plainly: the times the customer sees are displayed in **their own device’s time zone**, not necessarily your store’s time zone. If a customer is browsing from a different time zone than your business, the times they see are automatically converted to their local clock (their browser’s time zone is also sent to BillionBiz behind the scenes). This is normally the least-confusing behaviour, but it means a customer in a different time zone from your store is never shown “your” local time directly.

Screen states — expected results:

State	What the tester should see
Loading availability	Four pulsing placeholder bars in a two-column grid where the times will appear
Error loading availability	“We couldn’t load availability right now. Please try again.”
Empty day (no slots that day)	“No times available on this day. Try another date.”
Slot successfully held	Countdown bar appears (“Holding your slot — mm:ss”); a “Continue” button appears
Hold about to expire	Countdown turns red/bold in the final 60 seconds; text still shows the ticking time
Hold expired	“Hold expired” on the bar, toast “Your held slot expired. Please pick a new time.”, Continue button disappears

Edge case — someone else takes the slot the instant your customer taps it: the tap is rejected by the server. The tester should see a plain error line under the picker with the server’s own rejection message (this is a generic error banner, not a dedicated “slot taken” message with a “pick another” button — that friendlier treatment only exists one screen later, on Confirm, for the second collision check described below). The slot simply does not get held; the customer stays on the same screen and can tap a different time.

Filling in your questions and contact details

Next, the customer fills in any custom questions you’ve set up for that service (short text, a paragraph, single choice, multiple choice, checkbox, a number, or a date/time/date-time — whichever question types you’ve configured), each marked as required or optional.

If the customer is signed in, their name and phone number are simply shown back to them (read-only) — nothing more to type here, and no channel choice (see below) either.

If the customer is booking as a guest (not signed in), they’re additionally asked for: - Their full name - Their phone number - **How they’d like to receive their booking link** — a choice of WhatsApp, Email, or Both. WhatsApp is the default if they don’t change it. If they choose Email or Both, an email address becomes required and is checked for a valid format.

Validation messages — exactly what a tester should see:

Field left blank / invalid	Message shown
A required custom question (text/paragraph/ etc.)	“Required”
A required checkbox question, left unchecked	“This must be checked”
A required multiple-choice (“select several”) question, none picked	“Select at least one option”
A required email-type custom question, invalid format	“Enter a valid email”
Guest’s name, left blank	“Please enter your name”
Guest’s phone, too short/too long (valid range is 6–20 characters)	“Please enter a valid phone number”
Guest’s email, required (Email/Both chosen) but left blank	“Please enter your email address”
Guest’s email, filled in but not a valid format	“Please enter your email address” is used for blank; “Please enter a valid email address” for a malformed one

The message appears inline, directly under the field, the moment the customer tries to continue with it wrong.

Reviewing and confirming

The customer sees a summary — service, staff/resource (if any), date and time, and the **full service price** (not a deposit amount — see below) — with a final “Confirm booking” button. The 5-minute hold countdown is still visible and still ticking here; if it runs out on this screen, the same expiry-and-restart behaviour applies.

- If **no deposit is required**, the customer sees: “No payment is required now — you’ll pay at the time of your appointment.”
- If **a deposit is required and the customer is signed in**, they’re told: “A deposit is required to confirm this appointment. We’ll give you a link to pay it on the next screen — your slot is held until then.” — and tapping Confirm takes them straight into the in-app deposit-payment screen (see below).

- If **a deposit is required and the customer is a guest** (not signed in), they see the exact same deposit notice as above — because guests don't pay inline during the flow itself. Their booking is still created, but marked pending until that deposit is paid via their link afterwards.

No deposit amount is shown anywhere on this screen for either signed-in or guest customers — only the fact that one is or isn't owed. The exact figure only appears one screen later, fetched fresh from the server.

Screen states — expected results:

State	What the tester should see
Confirm in progress	Confirm button disabled while the request is in flight
Slot taken by someone else (checked again on submit)	"Sorry, this time was just taken by someone else." plus a "Pick another time" button that clears the hold and returns to the date/time screen
Any other confirm failure	"We couldn't confirm your booking. Please try again." (or the server's own message, if one came back)
Hold expires while reviewing	Same expired-hold bounce-back as the slot-picker screen
Success, no deposit owed	Proceeds straight to the confirmation screen
Success, deposit owed, signed in	Proceeds to the deposit-payment screen
Success, deposit owed, guest	Proceeds to the confirmation screen (pending-payment version) — never the payment screen

Deposits and paying

Whether a deposit applies, and how much it is, depends entirely on how you've priced that service — it is not something the customer chooses, and the customer never sees or influences the number before this screen.

- **Signed-in customers** who owe a deposit are taken straight into a payment screen as part of the booking flow, before they see the "Booking confirmed" screen. Page title: "Pay deposit"; heading: "Reserve your booking"; subheading: "A deposit is required to secure this appointment. You'll pay the remaining balance later."; the amount is labelled "Deposit due."
- **Guests** never see an in-app "pay now" screen during the flow itself. Their appointment is created right away in a pending-payment state, and they're given a link (see "What confirmation the customer receives," below) they can open at any time afterwards. Opening that link and tapping "Pay deposit" lands them on the exact same payment screen a signed-in customer would use — a guest does not need to sign in to pay.
- **A customer settling a remaining balance later** (after the deposit was already paid, at a different time — reached from "My Bookings" or from the guest link, never from the middle of the booking flow) sees the same screen with different wording: page title "Pay outstanding balance"; heading "Complete your payment"; subheading "There's an outstanding balance on this booking."; amount labelled "Balance due."

The amount shown is never calculated by the app itself. Every single time this screen loads — including a customer returning after abandoning it earlier — the app asks BillionBiz's server "what is currently owed on this booking" and shows exactly that answer, in the currency the server returns. It is not a number stored from an earlier visit, and it is not derived from the service's price on the customer's device.

Edge case — the balance is requested but the deposit itself was never actually paid: the server rejects the balance request; the app automatically retries as a deposit request instead and shows the deposit amount/copy, so the customer is never stuck on a balance screen for money that was never actually due yet.

Edge case — the customer tries to pay something that's already fully paid (leg already settled, e.g. they use an old "pay balance" link a second time after already paying): the server says nothing is owed; the app shows no Pay button and no error at all — it silently sends the customer onward (to the confirmation screen, My Bookings, or the manage page, depending on how they arrived).

Screen states — expected results:

State	What the tester should see
Amount loading	No amount shown yet (“—”); Pay button disabled
Amount ready	Exact amount shown; Pay button enabled, reading “Pay deposit” or “Pay balance”
Payment window open	Button reads “Processing…”
Nothing currently owed on this leg	No error, no Pay button — customer is redirected onward automatically
Payment order couldn’t even be started	“We couldn’t start the payment. Please try again.” with a Try again button
Payment window couldn’t load at all	“Payment couldn’t load. Please refresh and try again.”
Card/payment declined, or window closed without paying (abandoned)	A short, friendly decline message; the Pay button is simply available again — nothing about the booking is cancelled, and no new charge is made until the customer taps Pay again
Payment appeared to go through but the server couldn’t confirm it	“We couldn’t confirm your payment. Please contact us if the amount was deducted.”
This booking’s payment genuinely can’t be completed online	“This booking’s payment can’t be completed online right now. Please contact us for help.”

Abandon and return: if a customer closes the payment window (or the whole tab) partway through and comes back later — from the confirmation screen’s link, from My Bookings, or from their guest manage link — they land back on this same Pay screen, which re-fetches the current amount owed fresh. Nothing about the booking or the hold is penalised by walking away from a payment attempt at this stage (the appointment itself already exists by the time this screen can be reached — the 5-minute slot hold from earlier is no longer a factor here).

Confirmation — and an important distinction

Once confirmed, the customer lands on a confirmation screen showing the service, date/time, and total.

This is the one rule worth stating clearly: a booking that still has a deposit outstanding is never shown to the customer as "confirmed." Concretely:

	Deposit already paid (or none owed)	Deposit still owed
Page title	"Booking confirmed"	"Booking pending payment"
Heading	"You're all set!"	"Almost there!"
Subheading	"Your appointment has been booked. We look forward to seeing you."	"Pay your deposit to confirm this appointment."

This distinction is identical whether the customer is signed in or a guest.

- **Signed-in customers** additionally see a "View my bookings" button as well as a "Book another" button.
- **Guests** are shown a box with a personal link to save, with its own copy branching the same way:
  - Nothing owed: heading "Save your booking link", body "Save this link to view or manage your appointment later."
  - Deposit owed: heading "Pay your deposit to confirm", body "Use this link to pay your deposit — your appointment is held but not yet confirmed."
  - A "Copy" button sits next to the link (turns to "Copied" briefly after use). This link is the guest's only way back into this specific booking later — there's no account to log into.

Edge case — reopening the confirmation screen without the data it needs (a hard refresh, or a bookmarked/shared bare confirmation URL with no details behind it — by design, there's no way to fetch someone else's confirmation details from just an id, to prevent guessing at other customers' bookings): the customer sees a generic fallback notice instead of the full breakdown: "Your booking was confirmed. Check your messages for the confirmation details."

Edge case — reopening a confirmation a second time after a retried request already used up the one-time link: the save-box instead reads "This booking is already confirmed. Use the manage link you were sent to view or update it." (the raw link can only ever be shown once; if it can't be recovered, the customer is pointed back at whichever link they already have).

What confirmation the customer receives

Based on what's built into the storefront app itself, the customer's take-away after booking is: - The on-screen confirmation described above, at the exact status it's actually in (never shown as "confirmed" early — see above). - For a guest, the personal manage/pay link (shown on-screen, and copyable) — sent onward via WhatsApp, Email, or Both, according to whichever channel they picked on the details step. (No email is asked for at all if the guest sticks with the WhatsApp default.)

The storefront does not show the customer any separate in-app notification-bell entry for a new booking. Whether an additional automatic message (SMS, WhatsApp, or email) also actually goes out from BillionBiz's back-end messaging system, beyond the app displaying the link and recording the customer's channel choice, could not be confirmed from this app's own source — see Gaps, below.

Managing a booking afterwards

Signed-in customers find all their bookings under **My Orders → Bookings** (only visible once your store has appointments turned on). The list is split into "Upcoming" and "Past," and shows the service name, who it's with (or "Any available"), the date/time, and a status label: Pending payment, Confirmed, Checked in, In progress, Completed, Cancelled, or No-show. If money is still owed on a booking, that row also shows "Still owed: [amount]" and a "Pay deposit"/"Pay balance" link straight to the payment screen. Tapping a booking opens its full detail page. The list paginates ("Previous"/"Next", "page X of Y").

Screen states — expected results:

State	What the tester should see
Loading	"Loading your bookings…"
Error	"We couldn't load your bookings right now. Please try again."
Empty (never booked)	"You haven't booked anything yet." / "Once you book an appointment, it'll show up here." with a "Book an appointment" button back to `/book`
Detail page not found	"We couldn't find that booking." with a link back to the list

Guests use their personal saved link (`/book/manage/...`) instead of a login. It opens the exact same detail view a signed-in customer would see for their own booking — same information, same actions — just reached without an account.

Screen states — expected results:

State	What the tester should see
Loading	"Loading your booking…"
Invalid, malformed, or unrecognised link	"This booking link isn't valid." / "The link may have expired or been mistyped. Double-check the link we sent you, or contact us for help." — with a "Book a new appointment" button. Note: this exact same message is shown for EVERY kind of bad token (wrong, garbled, or simply unknown to the server) — the app deliberately can't tell a tester which of those it was, so nobody can use this page to guess at other customers' valid links. Whether a manage link ever actually expires on a fixed timer (as opposed to just being wrong) could not be confirmed in source — treat this as UNCONFIRMED and test with a genuinely invalid token instead.
A real connection problem (not a bad link)	A visibly different screen: "We couldn't load your booking." / "There may be a temporary connection problem. Check your connection and try again." with a "Try again" button

The money on a booking

Both the signed-in detail page and the guest link show the same breakdown, all pulled live from the server's own payment/refund records (never a stored estimate):

Row	Shown when	Exactly what it means
Service price	Always	The service's listed price — this is deliberately labelled "Service price," not "Total," since a deposit, discount, or add-on can make the actual charge different
Paid	Always, even when it's ₹0	Everything actually collected so far, so a customer can always answer "did my deposit actually go through?"
Still owed	Only when something is genuinely still due	The outstanding amount — never reduced by a refund still in progress
Refunded	Only when a refund has landed	Includes a note naming the amount actually retained, if the refund was partial (per your cancellation policy)
(Refund in progress)	While a refund is being processed	"Your refund is being processed. It usually reaches your original payment method within 5–7 working days."
(Refund failed)	If an automatic refund attempt failed	"We couldn't complete your refund automatically. Please contact the business." (shown as a clear alert, not a quiet note)
(No money block at all)	A booking with no payment record	"No payment was required for this booking."

Cancelling

A "Cancel booking" button is shown, but is only clickable when BillionBiz's server currently allows it right now — the app itself never guesses at your cancellation policy, and it can only ever take away a button the server already allowed (it never turns a blocked button into an

allowed one on its own). When cancelling is not allowed, the button is disabled and a reason is shown right next to it — never a bare greyed-out button with no explanation:

Reason shown to the customer	When
“Bookings can only be cancelled online more than {{X}} hours before the start time. Contact the business via WhatsApp if you need help.”	Outside your cancellation window
“This booking is already cancelled.”	Status is already Cancelled
“This booking was marked as a no-show. Contact the business if that’s wrong.”	Status is No-show
“This booking is complete.”	Status is Completed
“This booking has already started, so it can’t be moved. Contact the business if you need help.”	Status is In progress (reschedule uses the same wording; cancel is blocked too)
“Finish paying for this booking before changing its time.”	Deposit still owed (this exact line is worded for reschedule; the same “awaiting payment” block applies to cancel)
“Cancellation availability is being confirmed. Contact the business via WhatsApp if you need help.”	The policy itself failed to load — treated as blocked, never as “allowed”

If cancelling is allowed, tapping it opens a confirm dialog: - Title: “Cancel this booking?” - Body: “This can’t be undone. Any refund follows the business’s cancellation policy.” - Buttons: “Yes, cancel booking” / “Keep booking”

Edge case — the booking changed underneath the customer (e.g. it got cancelled or paid from another tab a moment earlier): the server rejects the action, both Cancel and Reschedule buttons grey out together, and the customer sees “This booking changed while you were looking at it. We’ve refreshed it — please try again.” — the screen then re-checks what’s actually still allowed.

Edge case — a refund is already running for this exact booking: “A refund is already being processed for this booking. Please contact the business.”

Edge case — too many attempts in a short time: “Too many attempts. Please wait a moment and try again.”

Rescheduling

A “Change time” button works the same way — only clickable when your policy currently allows it right now, with the same reason-shown-when-blocked behaviour and the same table of reasons above. When available, tapping it opens a dialog: - Title: “Pick a new time” - A date picker plus a time list, restricted to times with the **same staff member/resource** as the original booking — a note under the picker says “Your booking stays with [that person’s name].” The customer is never offered a different staff member as part of a reschedule. - Buttons: “Confirm new time” / “Keep current time”

Screen states inside the reschedule dialog:

State	What the tester should see
No times available that date	“No times are available on this date. Try another date.”
The exact new time gets taken by someone else before the customer confirms	“That time was just taken. Please pick another.” — the dialog stays open, the stale pick is cleared, and fresh times are refetched so the customer can just try again
Any other rejection (stale booking, policy, rate limit, etc.)	Dialog closes; the same shared error messages as the cancel flow above apply, and both Cancel and Reschedule buttons downgrade together until a refresh lands
Success	Dialog closes, the booking detail updates to the new time

Edge case — attempting to reschedule outside the allowed window: the button itself is disabled before the customer can even open the dialog, with the window-closed reason shown next to it (see the table above) — there is no way to open the picker and then be rejected for this specific reason, since the button is blocked first.

What a customer cannot do yet

- There is no way to book more than one service in a single pass — each booking is for one service at a time. (The review screen has a small “what you’re booking” summary card that is visually built to support multiple stacked services later, but today it only ever shows the one.)

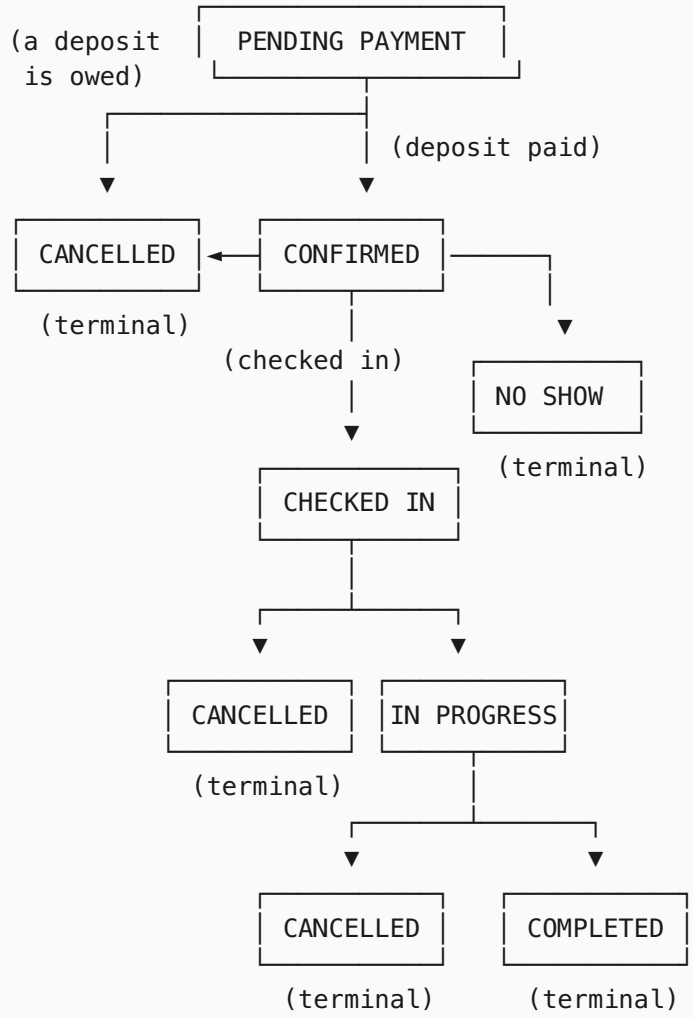
- The intake-question labels on the booking detail/history pages are shown using a simplified auto-formatted version of the question's internal name (for example, "serviceType" becomes "Service type") rather than the exact custom wording you typed when building the question — the exact answer is always shown correctly, only the label formatting is simplified.
- A customer cannot pick a specific staff member when rescheduling — only new times with the same staff member/resource as the original booking.
- A customer never sees a deposit's exact amount before the payment screen itself — even the review/confirm step only ever says whether one is owed, not how much.

Part 8 — The Life of a Booking — Statuses, Holds and Rules

This section describes the behaviour *behind* the Appointments screens — what happens when a slot is held, what money is owed and when, what messages go out and on which channel, and what happens when something is cancelled, is missed, or times out. It does not describe the admin or customer screens themselves (those are covered elsewhere in this guide).

The life of a booking

Every booking is one row that moves through a fixed set of states. A booking can only move to certain next states from wherever it is — there is no way to skip a step or move backwards.



Every booking is created as CONFIRMED first. A booking only ever visits PENDING PAYMENT if, at the moment the deposit is calculated, that deposit amount is greater than zero — the system then flips the brand-new booking back to PENDING PAYMENT in the same step that creates the payment request. A booking with no deposit configured (or a store that doesn't charge one) never touches PENDING PAYMENT at all — it goes straight to CONFIRMED and stays there.

Status	Meaning	Who/what moves it here	Can move to
Pending Payment	A deposit is owed and hasn't been paid yet	System, the instant a deposit amount > 0 is calculated for the booking	Confirmed (deposit paid), Cancelled (deposit never paid in time, or cancelled)
Confirmed	Booking is locked in; slot is held	System, right after a customer or staff member completes the booking	Checked In, Cancelled, No Show
Checked In	Customer has physically shown up	Staff/admin, manually	In Progress, Cancelled
In Progress	Service is under way	Staff/admin, manually	Completed, Cancelled
Completed	Service was delivered	Staff/admin, manually	*(terminal)*
Cancelled	Booking was called off	Customer, staff, or the automatic timeout sweep	*(terminal)*
No Show	Customer never arrived	Staff/admin, manually, from Confirmed only	*(terminal)*

Completed, Cancelled and No Show are dead ends — nothing can move a booking out of them again. Once a booking reaches Cancelled or No Show, its time slot is immediately given back to the calendar; every other status (including Pending Payment) still holds the slot.

A staff member can move a booking through Checked In → In Progress → Completed, or cancel it, at **any point with no notice-period restriction whatsoever** — staff/admin cancellation and rescheduling face only the rule table above, never a deadline. A **customer** cancelling or rescheduling their own booking faces an *additional*, separate permission check — see **Rescheduling and cancellation rules** (Part 8).

State-transition table (tester-grade — every event, every status)

This is the same lifecycle as **The life of a booking** (Part 8), expanded to show every event a booking can receive in each status — including the events that are **refused**, and the exact response a caller gets when refused. "Refused" means the system makes no change at all: no status change, no money movement, no notification.

Every refusal of an out-of-order status change comes back as a 400 with the message **"Cannot transition appointment from `<current>` to `<target>`."** and a stable code, `INVALID_TRANSITION` — this is the single generic guard behind every cell marked REFUSED below unless a more specific message is given. Rescheduling has its own, separate guard on top: **"Only confirmed appointments can be rescheduled."** (same code), which fires for any status other than Confirmed, before the generic guard is even reached.

Status	Event	Result	Money effect	Notification
Pending Payment	deposit is paid in full	→ **Confirmed**	order flips to “partially paid” (deposit-split) or fully paid (full-charge); balance leg becomes payable if anything remains	“confirmed” fires
Pending Payment	deposit-timeout sweep fires (see **Timer and background-job behaviour** (Part 8))	→ **Cancelled**	nothing captured yet, so no refund record is created	“cancelled” fires
Pending Payment	customer/staff cancels manually	→ **Cancelled**	same as above (nothing captured)	“cancelled” fires
Pending Payment	check-in / start / complete / no-show attempted	**REFUSED**	none	none
Pending Payment	reschedule attempted	**REFUSED** — “Only confirmed appointments can be rescheduled.”	none	none
Confirmed	staff checks the customer in	→ **Checked In**	none	none (no event type exists for this)
Confirmed	cancelled	→ **Cancelled**	cancellation-window forfeiture rule applies (**How the money moves** (Part 9))	“cancelled” fires
Confirmed		→ **No Show**		“no_show” fires

Status	Event	Result	Money effect	Notification
	marked as no-show		no-show forfeiture rule applies (**How the money moves** (Part 9))	
Confirmed	rescheduled (staff any time; customer if inside the notice window and no deposit owing)	stays **Confirmed**, new time	none	none (no reschedule event type exists)
Confirmed	start-of-service (in_progress) attempted directly, skipping check-in	**REFUSED**	none	none
Checked In	staff starts the service	→ **In Progress**	none	none
Checked In	cancelled	→ **Cancelled**	cancellation-window forfeiture rule applies	“cancelled” fires
Checked In	no-show attempted	**REFUSED** — no-show is only reachable directly from Confirmed	none	none
Checked In	reschedule attempted	**REFUSED** — “Only confirmed appointments can be rescheduled.”	none	none
In Progress	staff marks it complete	→ **Completed**	none forced — an unpaid balance is not	none (no event type exists for completion)

Status	Event	Result	Money effect	Notification
			auto-collected on completion	
In Progress	cancelled	→ **Cancelled**	cancellation-window forfeiture rule applies	“cancelled” fires
In Progress	check-in or no-show attempted	**REFUSED**	none	none
Completed / Cancelled / No Show	any event at all	**REFUSED** — every one of these three is a dead end	none	none

Two things worth a tester’s specific attention, both confirmed rather than assumed: - **A booking can go from Confirmed straight to No Show** with no check-in step required — but once it HAS been checked in or started, no-show is no longer reachable at all (the only two ways out of Checked In/In Progress are completion or cancellation). - **Reschedule is gated to Confirmed only**, for both staff and customers — a Pending-Payment, Checked-In, or In-Progress booking cannot be rescheduled by anyone, only cancelled or (for Pending Payment) paid forward into Confirmed first.

Holds and the booking time limit

Before a booking exists at all, picking a time slot creates a **hold** — a temporary, 5-minute claim on that slot while the customer finishes filling in their details and (if needed) verifies their phone number and pays.

- **Duration: exactly 5 minutes** from the moment the slot is picked.
- A hold is **advisory, not a hard lock**. It is used to keep the same slot from being *offered* to a second customer while the first one is mid-booking, but the database’s real double-booking protection (see **Capacity and overlap** (Part 8)) is what actually stops two people from being confirmed into the same slot — the hold by itself cannot do that.

- If the 5 minutes run out before the customer finishes, the hold simply expires and the slot becomes available to anyone again — there is nothing for the customer to see or be told, because by then they are still on the booking form; if they try to finish after expiry, the system tells them the hold has expired and asks them to pick a time again.
- **The 5-minute hold has no automatic reminder or notification of its own** — it is a quiet, short-lived row with no notification tied to its expiry.

This is a completely separate timer from the deposit-payment timeout below — don't confuse the two. The hold is about finishing the booking FORM; the deposit timeout is about finishing the PAYMENT after the booking already exists.

Deposit payment timeout (the real "time limit" merchants care about)

Once a booking exists and owes a deposit (status: Pending Payment), the merchant's own **deposit timeout** setting (default: 30 minutes, configurable per store) is the real clock. If the deposit still isn't paid once that many minutes have passed since the booking was created, an automatic sweep **cancels the booking outright** and frees the slot — a customer who never pays does not sit in limbo forever holding a slot hostage.

This sweep genuinely runs, automatically, every 10 minutes, as a real scheduled job — it is not a manual or theoretical process. It goes through the exact same cancellation path a staff member's manual cancel would use, so whatever cancellation/refund rules apply to a normal cancel (**Rescheduling and cancellation rules** (Part 8)/**How the money moves** (Part 9)) apply here too (in practice there is nothing to refund yet, since the deposit was never paid). If a customer's payment and the sweep race each other, the database's own row-locking guarantees only one of them wins — a payment that lands a split-second before the sweep reaches that row is honoured, never silently lost.

A parallel **automatic reminder sweep** runs every 5 minutes and separately **payment-link expiry housekeeping** runs once a day — both described in **Notifications** (Part 10).

Timer and background-job behaviour

Three time-based behaviours are supposed to happen with no human involved. All three run inside the same scheduling component (a jobs service separate from the main booking/order services), and depend on that component actually being deployed and running — none of this happens by magic inside the booking or order services themselves.

Behaviour	Schedule	What must be running	Symptom if it is NOT running
Deposit-timeout auto-cancel — an unpaid Pending Payment booking is cancelled once the store's deposit-timeout window (default 30 min) has passed	every 10 minutes	the scheduling component, AND it must be able to reach the booking service's internal endpoint	Pending Payment bookings never auto-cancel — a slot stays held indefinitely past its timeout. **A tester seeing this must first confirm the scheduler is actually running before filing it as an application bug** — it is the single most common false-positive in this module.
Reminder sending — reminders fire at each configured "minutes before appointment" mark (default 24h and 1h before)	every 5 minutes	the scheduling component, AND the notification/WhatsApp integration reachable	No reminder messages ever arrive, even though the booking itself is completely fine and Confirmed. Not a booking defect — a delivery/environment gap.
Payment-link housekeeping — reconciles outstanding payment links against actual gateway status	once a day (rides the existing daily payment-reminder run)	the scheduling component, AND the commerce service's sweep endpoint	Links that were quietly paid outside the normal webhook path may sit "unreconciled" longer than a day. This does NOT affect what a leg's amount-due actually computes to (**the money test matrix** (Part 12)'s live recalculation still works) — it only affects how quickly a stale-looking link gets flagged.

A comment in the scheduler's own source notes it currently runs as a **single instance with no leader-election** — if two copies were ever run at once, both could try to fire the same job at the same tick. This is not a live defect: the sweeps themselves claim their work with a row lock before acting (**Holds and the booking time limit** (Part 8), **Notifications** (Part 10)), so even a double-fire cannot double-cancel a booking or double-count a reminder — the safety net is in the sweep logic, not in the scheduler.

For anyone testing this module in a local or partial environment: if the scheduling component is not part of that environment's deployment, every row in this table will look "broken" (holds never expire past their timeout, no reminders ever send) purely because nothing is running the clock — that is an environment gap to note, not a defect to file.

Capacity and overlap — how double-booking is prevented

Two different booking shapes exist, and each is protected differently:

- **One-to-one services** (a haircut with a specific stylist, a doctor's appointment): only **one** booking can occupy a given staff member's time slot. This is enforced at the database level as a hard constraint — it is not just an application-level check that could theoretically be raced past; two simultaneous booking attempts for the same person/slot cannot both succeed, no matter how close together they happen.
- **Capacity services** (a fitness class, a workshop, a venue slot): a slot has a configured maximum number of seats. Bookings against it are counted, and a hard database check similarly refuses to let the count exceed the configured capacity, even under concurrent booking attempts.

Buffers: each service can carry its own prep/processing time and a buffer before/after the actual appointment. When calculating how long a booking actually blocks a calendar, the system adds together the base service duration, any processing time, and the buffer — so a 30-minute haircut with a 10-minute buffer genuinely reserves 40 minutes on the calendar, not just the visible 30.

Remaining capacity IS shown to customers for capacity-model services — availability search results carry a remaining-seats figure per time slot for that kind of service, computed live from the actual booked count, not a stale cached number. For one-to-one services there's no "capacity" concept to show — a slot is either open or it isn't.

Staff/resource constraints: a service can be restricted to specific staff members (or store equipment/rooms). If a customer doesn't pick a specific person, the system searches across every eligible staff member and offers "any available" as a booking option; the actual staff member is only locked in at the moment the booking is confirmed.

What's honestly unclear here: whether admin/staff schedule views surface a live "how full is today" figure beyond the day-summary numbers (booked count, unsold minutes, per-resource utilisation) already covered elsewhere in this guide — this section only speaks to the booking-time capacity check, not every possible admin reporting view.

Rescheduling and cancellation rules

Two entirely separate permission systems exist, and they are not the same rule applied twice:

1. **Staff/admin cancel or reschedule**: no notice-window restriction whatsoever. The only thing that can stop a staff member is the booking already being in a dead-end state (Completed, Cancelled, No Show) or the specific status-machine rule from **The life of a booking** (Part 8) (e.g. you can't "check in" a cancelled booking). A staff member linked to a specific resource can only act on that resource's own bookings, not everyone's.

2. **Customer self-service cancel or reschedule** (whether logged in, or a guest using their emailed/WhatsApp'd booking-management link): faces **both** the status-machine rule **and** a separate merchant-configured notice window (the "cancellation window", default 24 hours before the appointment, configurable per store/vertical — see **Deposit policy** (Part 9)'s preset table for vertical-specific defaults). A customer cannot self-serve past that deadline at all — after it passes, they would need to contact the merchant directly (there is no override on the customer side).

   The exact reasons a customer's self-cancel or self-reschedule button can be blocked, in order of priority:

   - the booking is already cancelled, already a no-show, or already completed (these always win, regardless of the notice window)
   - **for reschedule only**: the booking is checked-in or in-progress (you cannot reschedule an appointment that's already happening) — **note: a customer CAN still self-cancel a checked-in/in-progress booking**, this block only applies to reschedule
   - **for reschedule only**: the booking still owes a deposit (you cannot reschedule while payment is outstanding — cancel first)
   - the merchant's cancellation-window setting is missing or invalid for some reason — the system deliberately **refuses** self-service in that case rather than silently allowing it (fail closed, never fail open, when the policy itself can't be read)
   - the current time has already passed the calculated deadline (appointment time minus the notice window)

   If none of the above apply, self-service is allowed.

Known rough edge: no unhelpful-date-format error message was directly reproduced or found in the source read for this pass — the public availability-search endpoint's date validation returns a clear, specific error ("fromDate/toDate must be valid YYYY-MM-DD dates…") rather than a raw technical message. Any specific "unhelpful error" behaviour elsewhere in the booking or reschedule forms was not confirmed in source during this audit — treat as an open question rather than a confirmed defect, and record what you actually see during your acceptance pass.

Part 9 — Money — Deposits, Balances and Recording Payments

Deposit policy — what a merchant can configure

Every service a merchant offers ("offering") can have its own deposit policy, or fall back to the store's default. Four modes exist:

Mode	What the customer pays up-front
None	Nothing — the full amount is due later (or never collected online at all)
Percentage	A configured % of the service price
Fixed amount	A flat rupee amount, regardless of service price
Full prepay	100% of the price, collected as a single up-front charge

If an offering has no deposit policy of its own, the store's own default percentage is used automatically — a merchant does not have to configure every single service individually to have a working deposit policy.

Whatever the mode, the actual amount owed is a live calculation done from the service price at the moment of booking — never a number typed in and stored separately, so it always tracks the service's current price and cannot drift out of sync with it. This calculation happens once at the time the order is created; after that, both the deposit amount and the total do not silently change even if the merchant edits the service's price later — that booking's figures are locked in for that customer.

GST: each service can carry its own tax rate; deposit math is applied on the pricing engine's output, i.e. on whatever the customer is actually being asked to pay, so GST-inclusive/exclusive service pricing is respected rather than the deposit being computed against a pre-tax number that doesn't match what's shown on screen. *(Deposit amounts round to the nearest paisa, using a float-safe round-half-up applied at the first calculation — confirmed.)*

How the money moves (the most important part — read this section carefully)

A booking's money is never one lump sum. It is up to **two separate legs**:

- **Deposit leg** — the up-front amount, if the deposit policy calls for one.
- **Balance leg** — whatever is left once the deposit (if any) is paid.

If the deposit policy is "none" or "full prepay", there effectively isn't a second leg to chase — either nothing was collected up-front (balance/full amount is one single charge later), or everything was collected up-front in one shot.

The golden rule: money is never a stored number, it is always recalculated

How much a booking currently owes is never read off a saved column. Every time the system needs to answer "how much is still due on this booking," it adds up every successful payment ever made against that booking's order and every successful refund ever made against it, and subtracts. There is a number written down at the moment the booking is created (a "policy snapshot" of what the deposit and total were expected to be), but that number is a *reference point*, never the live answer — the live answer always comes from actually summing real payment and refund records. This is deliberate: a merchant can never see a due amount go stale or wrong just because a refund or an extra manual payment happened outside of the "normal" flow.

Which leg can be paid, in which state

A single, shared rule (used everywhere money-related decisions get made, so the two portals can never quietly disagree with each other) governs which leg is even eligible to be paid:

- The **deposit** leg is payable only while the order's payment status is literally "pending" and there is a deposit amount greater than zero on it.
- The **balance** leg is payable only once the order's payment status is "partially paid" (i.e. the deposit has landed) and there is still a balance greater than zero.

A deliberate difference between what the customer sees and what staff can do

There is one specific, **intentional** difference in how "what can still be paid" is answered, depending on who is asking:

- On the **customer's own booking page**, if a booking was minted for the *full* charge up-front (no deposit split) and is still unpaid, the customer is shown **no pay button at all** for that state — the system deliberately treats a not-yet-split full charge as "nothing for you to self-serve pay right now."

- On the **staff/admin side**, that exact same booking can still have a **payment link sent to the customer** by a staff member — admin tooling is allowed to collect that same full-charge amount by proactively pushing a link, even though the customer's own self-service screen won't offer it.

This is not a bug or an inconsistency to be fixed — it is a deliberate, documented product decision: self-service is intentionally more conservative than what a staff member is trusted to initiate on a customer's behalf.

If a leg is already paid, or the booking is in the wrong state

Both the "pay deposit" and "pay balance" actions are gated by the exact same state check above. If a customer (or a staff member acting on their behalf) tries to pay a leg that isn't currently eligible — because it's already been paid, because the booking never had that leg at all, or because the booking has moved past a payable state — the request is refused outright rather than silently accepting a duplicate or mismatched charge. The system also actively guards against ever collecting *more* than what is actually owed on a leg, independent of the eligibility check — an attempt to charge more than the computed amount due is rejected before any money moves.

Money collected "off-band"

If money is collected outside the normal pay-the-deposit / pay-the-balance flow (for example, a manual reconciliation, or a payment recorded against the order through another route entirely), the "how much is still due" answer updates automatically the next time it's asked — because, per the golden rule above, it is always recomputed from actual payment/refund records rather than trusted from a stored figure. Nothing needs to be manually "re-synced."

Cancellation and no-show — what happens to the money

Cancelling a booking that was never paid simply cancels it — there is nothing to refund, and no refund record is created at all in that case (the system doesn't create an empty/zero refund just for the sake of having one).

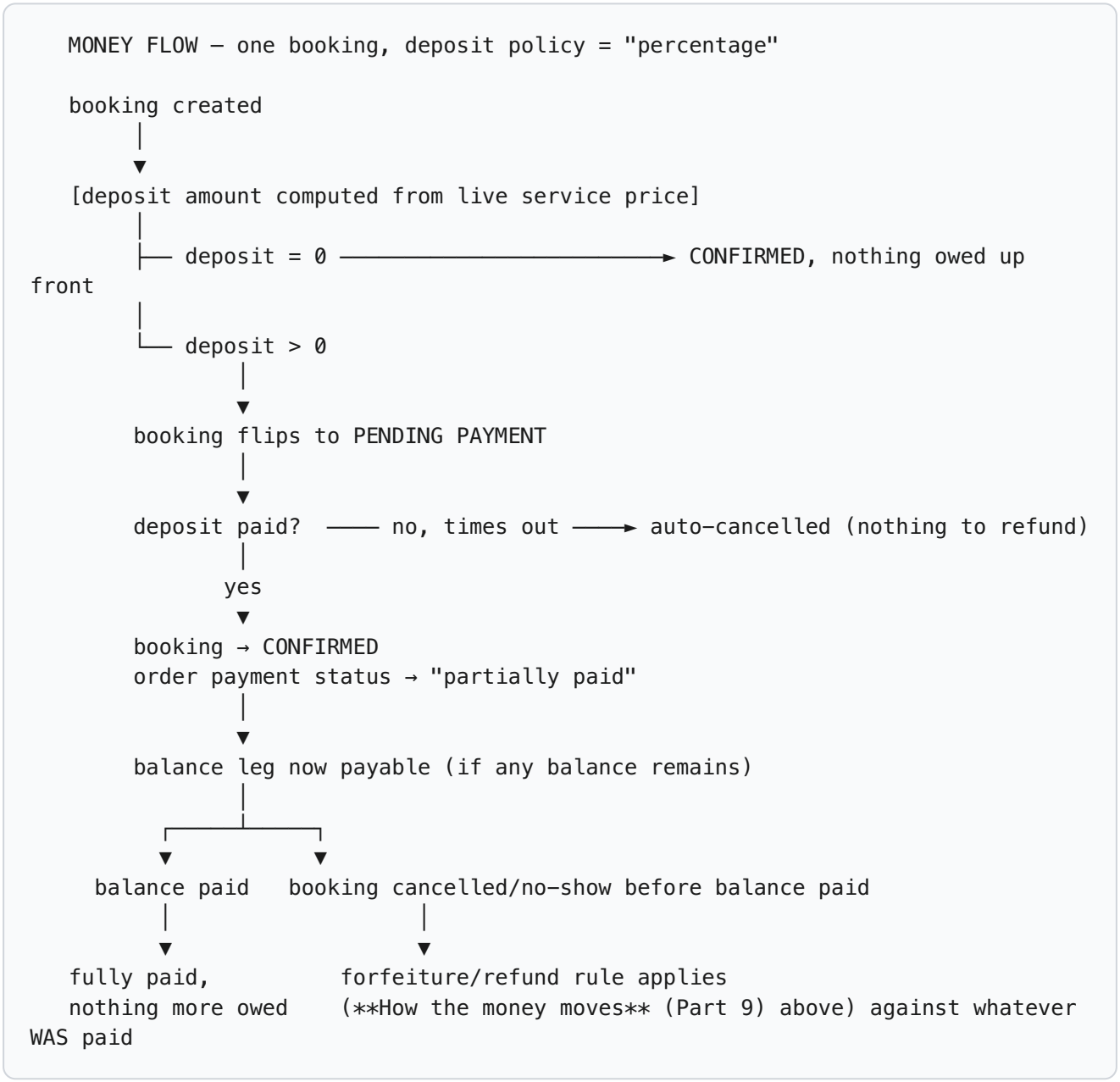
Cancelling a booking that WAS paid triggers real money-back logic, and the rule differs by *when* the cancellation happens relative to the merchant's cancellation-notice window (see **Rescheduling and cancellation rules** (Part 8) for what that window is):

- **Cancelled inside the notice window** (i.e., too close to the appointment, or after it's already passed): the customer forfeits **up to the deposit amount** — the deposit is kept, anything paid beyond the deposit is refunded.
- **Cancelled outside the notice window** (with enough notice): a **full refund** of everything paid.

No-show (the customer never turned up) uses a different, fee-based rule: the amount actually kept is **whichever is larger** of (a) the store's configured no-show fee for that booking, or (b) the deposit amount already paid — but never more than what was actually captured in the first place. In plain terms: a no-show always costs the customer at least their deposit (if they paid one), and can cost more, up to the configured no-show fee, but the merchant can never keep money that was never actually paid.

If a refund for a booking has already been processed once, trying to trigger it again is safely rejected rather than double-refunding.

What is NOT verified in source for this pass: whether refunds are instant/synchronous from the customer's point of view, or whether there is a visible in-between "refund pending" state shown on the booking screens themselves. The refund is recorded and then handed off for gateway processing; the on-screen customer-facing wording during that gap was not directly inspected in this pass. See GAPS.



Money in the admin panel — the Payment section

This is the section admins ask about most, so it's worth being precise.

The numbers on a booking

Inside the Payment section of a booking's detail panel:

Row	Meaning
Total	The full price of the booking.
Deposit	The deposit amount that applies to this offering's deposit policy, if any.
Balance due	The remaining amount after the deposit, as fixed at the time the booking's payment terms were set.
Paid so far	What has actually been paid against this booking to date.

Important nuance: "Balance due" is a fixed figure set once, at booking time, based on the offering's deposit policy — it does **not** recalculate itself as partial payments come in. The amount that actually still needs collecting right now is worked out separately from live payment records every time you open the booking, and *that* live figure — not the frozen "Balance due" — is what pre-fills the amount box when you record a payment. In practice this means: if a customer has already paid something toward the balance, the "Record payment" box will correctly suggest only what's genuinely still outstanding, even though the "Balance due" label itself doesn't visually update to match.

If nothing is due right now, the panel simply states: *"There's nothing due on this booking right now."*

Collecting payment

If a booking has no payment/order set up yet at all, you'll see a single **Collect payment** button to start one. Once an order exists, two actions sit side by side (never both active at once — an active payment link and manual recording are mutually exclusive at any moment):

Record payment — opens a dialog to log a payment you took another way (cash, card machine, etc.) directly against the booking. The amount field is pre-filled with the live outstanding amount described above, and can be adjusted.

Send payment link — opens a dialog to generate a payable link for the customer:

- A summary box shows exactly which amount the link will collect — the deposit if one is still outstanding, otherwise the balance.

- **Send via** — a choice of channels:
  - **Copy link** and **Email** are always available.
  - **WhatsApp** is available when your store's WhatsApp connection is set up; otherwise it's not offered.
  - **SMS** is always shown as unavailable — the exact copy reads *"Not available yet — SMS delivery isn't connected for this store."* SMS payment links are not a live feature on any plan today.
- **Link expires in (days)** — defaults to 1 day if a deposit is what's being collected, or 7 days if it's collecting the full balance; you can change it, but not to less than 1 day.

Once a link is created, its status is tracked and shown on the panel (Awaiting payment / Paid via payment link / Link expired / Link cancelled), and you can cancel an active link — with a confirmation, since the customer immediately loses the ability to pay through it.

Money section — states, validation, and expected results

Situation	What's shown
Payment details fail to load	Toast error: "Failed to load payment details"
An active payment link fails to load	Toast error: "Failed to load payment link"
Recording a payment with the amount field left blank/zero	Inline dialog error: "Enter an amount to record"
Sending a payment link fails	Toast error: "Failed to send payment link"
Recording a payment fails	Toast error: "Failed to collect payment"
Setting "Link expires in" to less than 1 day	Inline dialog error: "Enter at least 1 day"

Expected results for a tester: - Opening Record payment with the amount field cleared and clicking submit → blocked, "Enter an amount to record," dialog stays open. - Setting the payment-link expiry to `0` days and trying to create the link → blocked, "Enter at least 1 day." - Successfully recording a payment that brings the order to fully paid → the Payment section updates to show "There's nothing due on this booking right now," and (per Part 6) the booking's status may move toward Confirmed if it was previously Awaiting payment. - Cancelling an active payment link → after confirming, the link's status changes to "Link cancelled" and a new Send payment link action becomes available again.

GST and invoicing

An offering's GST rate and SAC code (Part 3) are captured for tax purposes on the service itself. As of this guide, the booking detail panel does not show a separate GST breakdown or generate a downloadable invoice document for a booking — see Part 13.

Part 10 — Notifications, Reminders and Timezones

Notifications

Every meaningful booking event can trigger an outbound message. The events are: **booking confirmed**, **reminder**, **cancelled**, **no-show**, **new booking alert to the merchant**, **one-time verification code (OTP)**, and **payment link**.

- **Channel**: the default channel for confirmation/reminder/cancelled/ no-show/merchant-alert/OTP messages is **WhatsApp**, using pre-approved message templates. **Payment-link** messages are the one exception — a customer can choose to receive their payment link over **WhatsApp, email, or both**; if they don't specify, WhatsApp is the default. There is no SMS channel for any of this.
- **Sending a WhatsApp message means it was successfully *submitted* to the WhatsApp send pipeline** — this system does not itself confirm the message was actually delivered to or read on the customer's phone; that is a property of the messaging integration downstream, not something tracked here.
- **A failed send on one channel never blocks another channel or blocks the booking itself.** If a payment link fails to send by WhatsApp and by email, the booking and the debt still exist exactly as before — the customer can still find and pay via the confirmation screen or their manage-booking link; a failed notification is a delivery problem, never a booking problem.
- **The one-time verification code (OTP)** is sent to a guest's phone at the moment they place a hold on a slot — before any real booking row even exists yet. As of this audit, **verifying the OTP is not a hard requirement to actually complete a booking** — it exists in the flow, but a guest can currently finish booking without having successfully verified their code. This is a stated, deliberate, temporary state (a harder gate is planned as a future, explicitly separate decision) rather than an oversight — but merchants should know it today, an OTP send failure or skip does not, by itself, stop a guest from booking.
- **Reminders** are sent according to a store-configured cadence — a list of "how many minutes before the appointment" (default: 24 hours before, and again 1 hour before; the vertical presets in **Deposit policy** (Part 9) vary this, e.g. clinics add a 2-hour reminder, class/venue bookings add a 3-day-ahead reminder on top). Each cadence point is only ever sent once per booking (a durable record is kept so a restart or a slow moment can't cause a duplicate reminder), and reminders only fire for bookings that are Confirmed or Checked In — a cancelled or completed booking never gets a reminder.

- **This reminder sweep genuinely runs automatically every 5 minutes**, as a real scheduled job, not a manual or theoretical process.
- **Payment-link housekeeping** (checking on the actual gateway status of outstanding links and flagging any that were paid without the system hearing about it through the normal channel) runs automatically **once a day**, riding along with the platform's existing daily payment-reminder sweep.

Confirmed finding, not a gap: the full set of things the system can ever send a notification about is fixed at exactly seven kinds — booking confirmed, reminder, cancelled, no-show, a new-booking alert to the merchant, the OTP code, and a payment link. There is no eighth kind. This means **check-in, start-of-service, completion, and reschedule do not send any customer notification at all, for anyone** — not because a message failed to send, but because no such message exists to send. See **Part 13 — What Isn't Finished Yet** and **Known limitations and open defects** (Part 13).

Timezones

Every store has its own configured timezone, and all availability calculations — which hours are open on a given calendar date, how a 15- or 30-minute slot grid lines up with the merchant's actual opening time, how a "minimum notice" cutoff is applied — are done in that store's timezone, not in server time or in the visiting browser's timezone. When a slot needs to be snapped forward to respect a minimum-notice rule, the snap is calculated from the merchant's own opening time on the grid, not from a generic clock-based rounding — this specifically matters for a timezone like India Standard Time that sits at a half-hour offset from UTC, where naive rounding would produce booking times that don't actually align with the merchant's stated opening minute.

A known historical class of bug, now specifically guarded against: the underlying timestamp columns used to be compared using the local timezone of whatever server process happened to be running the check, rather than always in UTC — which could cause a booking to be swept/cancelled too early depending on what timezone the server itself was running in. The current automatic deposit-timeout sweep does its comparison entirely in the database, using explicit UTC timestamps, specifically to avoid that failure mode recurring.

Part 11 — Plans, Limits and Staff Permissions

Plan entitlement — what unlocks Appointments

Appointments is a **plan-gated feature**, not something every merchant automatically has:

- It is **enabled** on the Growth, Max, and Pro plans.
- It is **disabled** on the Free and Starter plans — including Starter, even though Starter is a paid tier; this was a deliberate call, not an oversight.
- Beyond the on/off switch, the number of services ("offerings") a merchant can create is separately capped by plan: **10 on Growth, 50 on Max, unlimited on Pro** (Free/Starter can't reach offering creation at all, since the base feature itself is off for them).
- Staff seats for Appointments are tracked as their own add-on entitlement, separate from the base on/off switch.

How the system behaves when entitlement can't be checked, stated plainly, because the two failure modes behave differently: - If the merchant's subscription/plan information genuinely **cannot be fetched at all** (an outage, a data problem) — the system **fails closed**: it refuses the request rather than silently letting it through, unless it has a recent successful check cached from earlier to fall back on. - If the subscription loads fine but there's simply **no specific entitlement row configured** for a particular capability at all — the system's default behaviour in that narrower case is to **treat it as enabled/unlimited** rather than blocking it. This is why every plan tier, including the disabled ones, was deliberately given an explicit "disabled" row rather than just leaving the row out — an absent row is not a safe way to keep a paid feature switched off.

The plan-entitlement gate — exactly what happens at the boundary

Appointments is checked on every relevant request, not just once at login — including plain GET requests, which is stricter than most other gated features in the platform (many of those allow reads through regardless of plan; Appointments does not).

Situation	What happens	Denied or allowed?
Merchant's plan has Appointments explicitly disabled (Free/Starter, the normal case)	403, "Appointments is not available in your current plan", with a pointer to the billing/ upgrade page	**Denied**
The system cannot tell which store is asking at all (an internal identity problem on a write)	400, refused	**Denied** (fails closed)
The system cannot reach the plan/subscription lookup at all, and has no recent cached answer to fall back on	503, refused, marked retryable	**Denied** (fails closed)
The plan/subscription lookup succeeds but comes back with no plan attached at all (a corrupted/incomplete account)	503, refused	**Denied** (fails closed)
Everything above succeeds, but there is simply no entitlement row configured at all for Appointments on that specific plan	request goes through as if the feature were fully enabled and uncapped	**Allowed** (fails open) — **this is the release-blocking case**

The last row is the one a release audit must specifically check for, because it produces **no error to catch it by symptom** — a merchant on a supposedly-blocked plan would simply have working Appointments with no warning anywhere. The system deliberately ships an explicit "disabled" row for every single plan tier (including Free and Starter) specifically to avoid ever landing in that last row for a real plan. **Before signing off a release, confirm every plan tier has an explicit Appointments entitlement row present — an absent row, not an error, is the failure mode to look for.**

Plan limits and upgrading, in the admin panel

Appointments itself sits behind your plan. If it isn't included, opening any Appointments screen shows an upgrade prompt instead of the page content.

Within Appointments, two things are commonly capped by plan:

- **Number of offerings (services)** you can create — hitting the cap shows an inline message on the offering form rather than a pop-up, with a link to upgrade or add capacity.
- **Staff seats** (Person-type team members) — as covered in Part 4, most plans start with **zero staff seats included by default**; adding staff generally requires purchasing seats as an add-on rather than something a plan tier unlocks automatically. The seat-usage banner and the team-member dialog both link straight to the add-ons billing screen when you're at or over your limit.

Exactly which named plan tier first unlocks Appointments, and the specific cap numbers for offerings/other limits per tier, are controlled by your subscription on the backend and are not fixed values inside the admin panel itself. Check your current plan's entitlements on your billing screen.

Part 12 — Testing the Module End to End

This part is a hands-on checklist for anyone signing off the Appointments module — a tester, a release manager, or a merchant doing their own acceptance pass before trusting it with real customers and real money. Follow the checklists in order; each one builds on data created by the one before it.

Before you start

Must exist	Symptom if it's missing
A store whose subscription passes the appointments plan gate	Opening **Appointments** in the admin panel shows an upgrade message instead of the calendar. On the storefront, the public booking API for that store answers "plan gate unavailable" rather than a service list.
At least one bookable person (a Team & Resources entry, type **Person**)	The New appointment form and the customer booking flow can offer no time slots at all — a service with nobody assigned to it "won't offer any time slots."
Working hours (at least one **Weekly shift**) set for that person	The resource "shows as never-available until a shift is added" — no slots appear for them anywhere, even though the person and the service both exist.
At least one published, active service (an **Offering**) with **Visible for online booking** switched on	The service doesn't appear in the storefront's service list at all (it still exists for staff-only in-person booking, but a customer browsing `/book` won't see it).
That service has at least one team member assigned under its **Team** field	Same failure as "no bookable person" above — the booking form and the storefront funnel offer no slots for that service.

Seed all five before running any checklist below — a SKIP because "there was no data to test with" is not an acceptable result for any row in this document.

Canonical money fixture used throughout Checklist C: one Offering named "Fixture Service," **Price ₹1200**, **Deposit type = Percentage of total**, **Deposit value = 30%**. That gives: - **Deposit = ₹360** - **Balance due (frozen at booking time) = ₹840**

Checklist A — Admin setup

#	What you do	What you should see	If you see something else
A1	Admin panel → left sidebar → **Appointments**, first visit with nothing set up yet	The **Settings** tab shows a "Getting set up" checklist with all four items unticked, plus a **Start from a preset** row of business-type chips	If any checklist item shows ticked before you've done the matching thing, the checklist is fabricating progress — log this as a bug, it should never happen
A2	**Offerings** tab → **Add offering** → fill Basics (Name = "Fixture Service"), set **Duration** = 30, **Price type** = Fixed, **Price** = 1200, **Deposit type** = Percentage of total, **Deposit value** = 30, turn on **Visible for online booking**, save	Offering saves, appears in the Offerings list as Active, "Add your first service" checklist item on Settings is now ticked	Save fails, or the checklist item stays unticked after a genuine save — log as a bug
A3	On the saved Fixture Service, open the **Team** field and add one staff member	Offering saves with that person listed	If you leave Team empty and try to book this service later, expect zero time slots (see Checklist E) — this is expected, not a bug, if you deliberately test it
A4	**Team & Resources** → **Add team member**, Type = Person, fill Name/Email/password, leave **Set an initial weekly shift** blank, save	New resource appears, Active, with no shift set	—
A5	Open that resource's **Availability** icon-link before adding a shift	The screen tells you this resource shows	If the screen shows nothing about this and just looks like an

#	What you do	What you should see	If you see something else
		as never-available until a shift is added	empty schedule with no warning, log as a bug (a merchant could easily miss why nobody can book this person)
A6	Add a Weekly shift: Monday 09:00–17:00 (pick a day that is “today” or “tomorrow” in your test timezone so Checklist B has slots to book), save	Shift appears in the Weekly shifts list; “Add a team member or resource” checklist item on Settings is now ticked	—
A7	Settings tab → set a default deposit percentage (30%) and a no-show fee percentage — leave it at the default (0%) for the canonical fixture — save	“Set booking rules” checklist item ticks; the value you saved persists on reload	—
A8	Settings tab → configure at least one reminder time, save	“Set reminder cadence” checklist item ticks	—
A9	Settings tab → click a business-type preset chip (e.g. “Salon”)	A confirmation dialog warns this replaces **all** currently saved booking rules and cannot be undone	If it applies without warning, or merges instead of overwriting, log as a bug — then re-do A7/A8, since this step is destructive by design
A10	Offerings tab → **Deactivate** the Fixture Service, then look at the list with **Active only** off vs on	With **Active only** on, it disappears from the list; with it off, it still shows, marked Inactive. There is no	—

#	What you do	What you should see	If you see something else
		delete option — only Edit/Deactivate	
A11	Re-activate the Fixture Service via Edit before moving to Checklist B	It becomes Active and bookable again	If you cannot find a way to flip it back to Active from the Edit form, log as a bug and use a fresh offering instead for the rest of this document

Checklist B — The customer booking journey

Run this as a signed-out guest first, then repeat once signed in (see the branch notes in each row).

#	What you do	What you should see	If you see something else
B1	Visit your storefront's `/book` page	A list of services, each showing photo/monogram, name, duration, and price — Fixture Service shows **₹1200**	A blank grey box instead of a photo/monogram, or a missing price — log as a bug
B2	Tap Fixture Service	If more than one eligible staff member exists, a staff-picker screen with an "Any available" option appears; if only one, this step is skipped and you go straight to picking a time	—
B3	Pick a date and time on the Monday shift you seeded in A6	The moment you tap a time, it's held — a countdown timer appears showing time remaining (e.g. "4:59 remaining")	No countdown appears, or the slot doesn't visibly lock — log as a bug (this is the mechanism that prevents double-booking during the flow)
B4	Wait until the countdown enters its final minute	The countdown turns red/bold	—
B5	Let the countdown run out completely (don't submit anything for 5+ minutes)	You're bounced back to the time-slot screen with a message that your hold expired	The form silently lets you continue past expiry, or throws a raw error — log as a bug
B6	Start again, pick a fresh slot, fill in any custom questions for the service	Required fields show an inline error if left blank or answered wrong; optional ones don't block you	—

#	What you do	What you should see	If you see something else
B7	**As a guest**, continue past the details step	You're additionally asked for your name, phone number, and "How they'd like to receive their booking link" (WhatsApp / Email / Both), with WhatsApp pre-selected	If you pick Email or Both and no email field appears/ becomes required, log as a bug
B8	**Signed in**, repeat B1–B6 with a logged-in test account	Your name and phone are shown back to you read-only — nothing to type	—
B9	Reach the review screen	Summary shows service, staff (if any), date/time, and total **₹1200** — since this offering has a deposit, expect a note that a deposit is due, quoted as **₹360**	If the review screen shows ₹1200 as the amount due right now instead of ₹360, that is the broken number — log as a bug (see Checklist C for the full money trace)
B10	Tap Confirm	**Signed-in:** you're taken straight into an in-app payment screen for ₹360 before you see any confirmation screen. **Guest:** the booking is created immediately in a pending state, and you're told a payment link is coming rather than being asked to pay inline	—

Checklist C — Money: deposits, balances, refunds

Uses the ₹1200 / 30% Fixture Service from above. Run each row against a **fresh booking** unless told otherwise — don't reuse a booking across rows, since payment state doesn't reset.

#	What you do	What you should see	If you see something else
C1	**Guest** books Fixture Service, reaches the review screen	Deposit quoted = **₹360** (30% of ₹1200)	**Broken build shows ₹1200 (the full price) instead of ₹360 — that is the discriminating failure for this row.**
C2	Guest confirms the booking	Booking created as **Pending Payment**; guest is shown a saved link worded as a payment link ("Use this link to pay your deposit"), not "Save your booking link"	If the wording says "no payment needed" while a deposit is clearly outstanding, log as a bug — this exact wording bug shipped once before
C3	Guest opens the saved link, pays	Payment screen shows the exact amount due, sourced live from the server: **₹360**	Amount shown is anything other than ₹360 (e.g. ₹1200, or ₹840) — log as a bug and capture the network response for the amount-due call
C4	Payment succeeds	Booking flips to **Confirmed**; order payment status becomes "partially paid"; the balance leg (₹840) is now payable	Booking stays Pending Payment after a successful payment, or jumps straight to fully paid — log as a bug
C5	Admin opens the booking's detail panel → Payment section	Rows read: **Total ₹1200**, **Deposit ₹360**, **Balance due ₹840**, **Paid so far ₹360**	—
C6	Admin clicks **Record payment** for the remaining balance	The amount field is pre-filled with the **live outstanding amount**: **₹840**.	**This is the discriminating cell for the whole section: a correct build pre-fills ₹840 (₹1200 total**

#	What you do	What you should see	If you see something else
			minus the ₹360 already paid). A broken build pre-fills ₹1200 — the full order total, ignoring what's already been paid — or leaves the stale, frozen "Balance due" figure instead of the live one. Either wrong number is a bug; this exact class of bug (record-payment defaulting to a stale figure) has been fixed before, so re-testing it is not redundant.
C7	Confirm the ₹840 payment	Payment section now reads **Paid so far ₹1200**; the "what's due right now" line reads *"There's nothing due on this booking right now."*	Panel still shows an amount due after full payment — log as a bug
C8	Open the Calendar Day view for the day this booking falls on	**Collected** on the day-money bar includes this booking's ₹1200 (assuming no other bookings that day complicate the total); **Due at counter** does not include it	—
C9	**New fixture booking.** Pay the full ₹1200 (deposit + balance), then **cancel it from inside the merchant's cancellation-notice window** (i.e., very	Customer **forfeits up to the deposit amount**: ₹360 is kept, and the remaining **₹840 is refunded**	Full ₹1200 refunded, or nothing refunded — log as a bug; also check the admin/ customer refund-status wording, which

#	What you do	What you should see	If you see something else
	close to or after the appointment time)		is an explicit gap (see below)
C10	**New fixture booking.** Pay only the ₹360 deposit, then cancel it **well outside** the cancellation-notice window	**Full refund**: the ₹360 already paid comes back in full, since this cancellation is outside the notice window	Deposit is kept anyway — log as a bug
C11	**New fixture booking.** Pay only the ₹360 deposit, then mark it **No-show** from the admin panel (with the store's no-show fee left at the default of 0%)	Forfeiture = `max(deposit paid, no-show fee)` = `max(₹360, ₹0)` = **₹360 kept**, **₹0 refunded** — nothing extra beyond the deposit is taken, because nothing extra was ever paid	Anything other than exactly ₹360 kept/₹0 refunded — log as a bug. If you raise the store's no-show fee above ₹360 (e.g. to ₹500) and repeat this row, the kept amount should rise to match the fee, but never above what was actually captured
C12	Try to trigger a refund a second time on the same cancelled booking from C9	Request is rejected — no second refund is created	A second refund is created (double-refund) — log as a **critical** bug
C13	Attempt to record an admin payment larger than the live outstanding amount on any booking with money still due	The system refuses the overcollection — money is not taken	Overpayment is silently accepted — log as a **critical** bug

Checklist D — Changing a booking: reschedule, cancel, no-show

#	What you do	What you should see	If you see something else
D1	Admin opens a **Confirmed** booking → **Reschedule** → pick the *same* date/time it's already booked for	The booking's own current slot is offered back as a valid option, even though the resource is otherwise fully booked at that time	Own current slot is missing from the offered times — log as a bug
D2	Admin reschedules a Confirmed booking to a genuinely different open slot	Booking moves to the new time; old slot is freed	—
D3	Admin cancels a booking at any status that allows it (Awaiting payment, Confirmed, Checked in, In progress) — **immediately**, with no notice-period wait	Cancellation succeeds regardless of how close the appointment is — staff/admin has **no notice-window restriction at all**	Admin cancel is blocked by a notice window — log as a bug (this would be backwards; only customers face that restriction)
D4	Admin checks in a Confirmed booking, then starts it, then completes it	Status moves Confirmed → Checked in → In progress → Completed, one step at a time, only the valid next action offered at each stage	Any button appears that isn't valid for the current status (e.g. "Check in" showing on a Completed booking) — log as a bug
D5	Admin marks a **Confirmed** booking as **No-show**	Status moves to No-show; slot is freed immediately	No-show offered from a status other than Confirmed — log as a bug
D6	**Signed-in customer**, on a Confirmed booking with no deposit outstanding, clicks **Cancel booking**	Button is greyed out; a reason is shown next to it (cancellation window has closed)	Cancel succeeds anyway — log as a **critical** bug (this is the customer-side deadline the whole

#	What you do	What you should see	If you see something else
	inside the cancellation window (e.g. same day, with the store's default 24-hour window)		business rule exists to enforce)
D7	Same customer, same booking, well **outside** the cancellation window	Cancel button is clickable; confirms via a "are you sure" dialog; cancellation goes through	—
D8	Customer attempts to **Reschedule** a booking that is still **Pending Payment** (deposit unpaid)	Reschedule is blocked with a reason — pay first, or cancel	Reschedule succeeds while a deposit is still owed — log as a bug
D9	Customer attempts to **Reschedule** a booking that is **Checked in** or **In progress**	Reschedule is blocked (you can't reschedule something already happening) — but note: **Cancel is still allowed** on a checked-in/in-progress booking, this block is reschedule-only	Reschedule succeeds on a checked-in booking, or cancel is also blocked — log as a bug
D10	Customer attempts self-service cancel/ reschedule on a booking that is already Cancelled, No-show, or Completed	Button greyed out / action refused, regardless of the notice window (these always win)	—

Checklist E — Limits, permissions, and negative paths

#	What you do	What you should see	If you see something else
E1	Log in as a store on a plan that does **not** include Appointments (Free or Starter)	Opening Appointments in the admin panel shows an upgrade prompt instead of the calendar	Calendar loads anyway — log as a **critical** bug (a plan-gated feature leaking through is a billing problem)
E2	On a Growth-tier store (10-offering cap), create an 11th active offering	Inline plan-limit message on the offering form, not a pop-up toast, with a link to upgrade/add capacity	11th offering saves without any limit message — log as a bug
E3	On a store already at its staff-seat limit, try to add another Person-type team member	Seat-usage banner reads “you’ve reached your plan’s staff limit,” with a **Buy seat** button linking to billing add-ons	Extra staff member is created past the limit — log as a bug
E4	Log in as a **staff** role user linked to a resource, navigate to Appointments	You land on **My Day**, with only **Calendar** and **My availability** visible — Offerings, Team & Resources, and Settings tabs are not shown	Staff user sees the full admin tab set — log as a **critical** bug (permissions leak)
E5	As the same staff user, try to directly open the Offerings or Settings URL rather than navigating via tabs	**UNCONFIRMED** — expected to be blocked the same way (route guard is per-screen, not just per-tab-visibility), but the exact blocked-screen behaviour for a direct URL hit was not confirmed in source for this document. Verify manually and	If it renders the full admin screen instead of blocking, log as a **critical** bug regardless of what this document predicted

#	What you do	What you should see	If you see something else
		record what actually happens	
E6	Delete/leave empty the **Team** field on an offering, then try to book it from the storefront	No time slots are offered anywhere for that service	Slots are offered despite no assigned staff — log as a **critical** bug (would create an unbookable-but-shown booking)
E7	Create a brand-new Team & Resources entry with **no** weekly shift, assign it to an offering, try to book that person	That person offers zero time slots; storefront shows "no one available" if they're the only option	Slots appear for a person with no shift configured — log as a **critical** bug
E8	Try to book a slot, then have a second browser/session book the exact same slot for the same one-to-one resource before the first finishes	One booking succeeds; the other is rejected with an error rather than both silently succeeding — this is enforced as a hard database constraint, not just a UI check	Both bookings succeed (a true double-booking) — log as a **critical** bug and capture both booking IDs
E9	Repeat E8 for a **capacity/group** service at exactly its configured capacity limit, with concurrent attempts totaling one over capacity	The count is refused past capacity, even under concurrent attempts	Capacity is exceeded — log as a **critical** bug
E10	With the merchant's cancellation-window setting somehow unset or unreadable (simulate if possible,	Self-service cancel/reschedule is **refused** (fails closed) rather than silently allowed	Self-service succeeds when the policy can't be read — log as a **critical** bug (this is a

#	What you do	What you should see	If you see something else
	e.g. a settings fetch error)		deliberate fail-closed safety rule)

Negative and edge paths

Scenario	Expected visible result	Covered above
Slot taken by someone else between choosing and confirming	Customer sees a message explaining the slot was just taken, with a button to go back and pick another time (checked again at the final Confirm step)	See B3–B5 flow; not separately numbered above — walk it by opening two sessions on the same slot
The hold/booking time limit expiring	Bounced back to the time-slot screen with an expiry message, must pick again	B5
A payment link that has expired	Admin panel shows the link's status as "Link expired." **UNCONFIRMED** what a customer sees if they open an expired *payment* link directly (distinct from the guest manage-booking link, which shows a generic "this link isn't valid" message) — verify manually	—
A payment that is declined or abandoned	Friendly error message with a way to try again; the booking/hold is not lost at this stage	See B10/Checklist C payment steps
Booking outside working hours	The time simply never appears as a selectable slot — no distinct error message is documented for this case	—
Booking inside the minimum-notice window	Too-soon times don't appear in the slot list at all (the grid snaps forward past the cutoff) — no distinct error message is documented for this case	—
Booking past the booking horizon	**UNCONFIRMED** — the customer-facing date strip only shows the next 14 days, which is narrower than the default 30-day horizon setting,	—

Scenario	Expected visible result	Covered above
	so this boundary may not be reachable through the visible UI at all in its current form. Verify manually whether a horizon violation is even reachable	
A staff member with no working hours set	That person offers zero slots; shows as never-available	A5, E7
A service with no eligible staff	Zero slots offered anywhere for that service	E6
A cancellation attempted after the cancellation window	Cancel button greyed out with a stated reason	D6
A reschedule to an already-taken slot	Customer sees an explanation and the picker refreshes with fresh times rather than failing silently	D1/D2 flow — repeat with a genuinely taken slot
A plan limit reached on creating a team member or a service	Inline plan-limit message (offerings) or seat-limit banner + Buy seat CTA (staff)	E2, E3
A staff-role user opening a management screen	Tab hidden from nav; direct-URL behaviour unconfirmed	E4, E5

What this checklist does NOT prove

Taken from the automated-coverage map audited for this release — do not assume anything below is covered just because a related area is green:

Covered by automated tests today (per the coverage map): the booking funnel end-to-end, deposit/payment quoting and recording, reschedule/cancel self-service and admin paths, the admin calendar and day-summary tile, store setup (offerings/resources/availability/staff-seat limits), and plan/feature gating. Running this checklist by hand is a *second*, independent check on top of those — it is not redundant with them, because it walks real screens with real clicks rather than calling functions directly.

Manual-only, not automated at all, right now: - The full cross-service money reconciliation walk — create a booking with a deposit → pay the deposit → confirm the invoice/outstanding-balance figures agree → pay the balance → confirm the booking status and the order's paid-status agree (Checklist C above). An automated version of this exists but is written on a branch not yet merged into the running test suite — this document's Checklist C is currently the only real check of this path. - Whether an actual SMS/WhatsApp/email notification is received on a real phone or inbox. Every automated test confirms the *system attempted to send* — none confirm delivery. If this needs proving, a human must check a real device. - Any true browser-level, end-to-end walk. No automated Playwright suite exists for appointments at all — this checklist, walked by a human in a real browser, is the closest thing to that coverage that currently exists for this module.

Cannot be verified today at all, and why: - Whether the backend service actually running behind the UAT API is the same commit this document describes — the backend publishes no version identifier reachable over HTTP, unlike the two front-ends. - The admin panel currently deployed to the UAT admin URL is significantly behind its own staging branch as of this writing — an admin-side walk of this checklist done against that stale build is testing older software than what this document describes, and the result should not be trusted for sign-off until a fresh deploy happens. Check the deployed build's own identity before relying on any admin-side result. - Exact rounding behaviour for a fractional-paisa deposit (e.g. 30% of an odd rupee amount) was not independently confirmed for this document — if your test prices produce a fractional-paisa deposit, treat the rounding as **UNCONFIRMED** rather than assuming standard rounding. - Customer-visible wording during the gap between a refund being recorded and actually landing (is there a visible "refund pending" state, or does the booking just show cancelled) was not confirmed — note whatever you actually see in C9/C10 above, since this document cannot tell you what to expect. - Whether check-in, start-of-service, or completion trigger any customer-facing notification (the same way confirm/cancel/no-show/reminder do) was not confirmed either way — do not assume silence here means "no notification" or "notification sent."

Money test matrix — worked example (₹1200 service, 30% deposit)

Fixed facts for every row below: service price = **₹1200**; deposit policy = percentage, 30% → **deposit leg target = ₹360**; **order total (netAmount) = ₹1200**. "Captured" = everything successfully paid against the order so far, from any source (online leg payment or an off-band manual recording), summed together. The system always computes what's still owed as `target − captured`, never off a number written down at the start — see **How the money moves** (Part 9)'s golden rule. This is what makes the matrix below testable: the same formula must produce every cell, regardless of how the money arrived.

#	Scenario (running total captured)	Deposit leg quote / eligibility	Balance leg quote / eligibility
1	Nothing collected (₹0)	**₹360**, payable	**Refused** — 409, "Booking order is not eligible for its balance payment" (deposit hasn't closed yet)
2	₹150 of the deposit collected off-band by the merchant (₹150)	**₹210** (360−150), payable	**Refused** — same as row 1; a partial off-band capture does not open the balance leg
3	Deposit exactly met (₹360, via a normal online payment)	**Refused** — 409, "Booking order is not eligible for its deposit payment" (deposit leg already closed)	**₹840** (1200−360), payable
4	Total partly collected — deposit met plus ₹200 of the balance (₹560)	**Refused** — same as row 3	**₹640** (1200−560), payable
5	Everything collected (₹1200)	**Refused** — same as row 3	**Refused** — 409, "This booking's balance payment has already been collected"

The one cell that separates a correct build from a broken one is a variant of row 2 → row 3, and it is a real, previously-shipped defect class, not a hypothetical: if the merchant collects **₹300 off-band** *before* the balance leg is ever quoted, and the customer separately pays the **₹360 deposit link**, total captured is **₹660** — the balance leg **must** quote **₹540** (1200 − 660). A broken build that instead quotes the balance leg's original ₹840 (a number fixed at the moment the booking was created, not updated for the ₹300 already collected) would let the store collect ₹300 + ₹360 + ₹840 = **₹1500 against a ₹1200 booking** — a real ₹300 over-collection. Testing this exact sequence — an off-band capture, THEN a request for the balance leg's amount — is the single highest-value regression check in the whole module.

A second guard exists specifically to make an over-collection like that impossible even if some future change reintroduces the bug: any attempt to collect an amount that would push total

captured past the order's ₹1200 ceiling is refused outright (409, "...would over-collect against the order total"), regardless of what amount was quoted.

UNCONFIRMED: exactly which of the two possible refusal messages fires in row 3 if the ₹360 is collected entirely off-band rather than through an online payment — this depends on whether an off-band recording alone ever flips the order's payment status the same way an online payment does, which was not independently verified in this session. Testers should run this exact variant and record which of "not eligible for its deposit payment" vs. "already collected" actually comes back — both are acceptable outcomes for **How the money moves** (Part 9)'s guarantees, but only one is the actual coded path today.

Rules and money flow checklist

Numbered end-to-end scenarios for the release test pass. Amounts assume the same ₹1200 / 30%-deposit service used in **the money test matrix** (Part 12) unless stated otherwise. Mark any step you cannot get a clean pass/fail on as `UNCONFIRMED` rather than guessing.

1. **Happy path (no-deposit service).** Preconditions: an offering with deposit mode "none". Steps: pick a slot → hold created → complete the booking with no payment step. Expected: appointment is created directly as **Confirmed** — it never visits Pending Payment. No leg is forced open by the booking itself.
2. **Deposit path.** Preconditions: ₹1200 service, 30% deposit. Steps: book the slot → appointment is **Pending Payment**, deposit leg quotes **₹360** → pay the deposit leg in full. Expected: appointment → **Confirmed**; balance leg now quotes **₹840**; deposit leg is now refused (already collected).
3. **Off-band capture, then the balance leg.** Preconditions: same booking as #2, deposit still unpaid online. Steps: merchant records **₹300** off-band against the order → later, request the balance leg's amount (once the order is in a state where the balance leg is eligible). Expected: the balance leg **must** quote **₹900** (1200−300) at that point if that ₹300 is the only money captured, or, if a further ₹360 was also captured online meanwhile, **exactly ₹540** (1200−660) — never the original ₹840. See **the money test matrix** (Part 12)'s regression case; this is the release-blocking check.
4. **Pay-in-full.** Preconditions: an offering with deposit mode "full prepay", ₹1200. Steps: book the slot. Expected: the booking mints for the full ₹1200 as a single charge with no deposit split; the customer's own self-service screen shows **no pay button** for it (**How the money moves** (Part 9)); staff can still send a payment link for the full ₹1200.

5. **Abandoned hold.** Preconditions: none. Steps: pick a slot, then stop — do not finish the booking form. Expected: after 5 minutes, the hold lapses and the slot becomes bookable by someone else; nothing is created, nothing is shown to the abandoning customer (they are still mid-form). `UNCONFIRMED`: whether the lapsed hold row itself is ever cleaned up versus just ignored going forward.

6. **Expired deposit-payment window.** Preconditions: ₹1200 service, 30% deposit, default 30-minute deposit timeout; the background scheduler (**Timer and background-job behaviour** (Part 8)) confirmed running. Steps: book the slot, do not pay the ₹360 deposit, wait past 30 minutes. Expected: within the next scheduled sweep tick (every 10 minutes), the booking auto-cancels; nothing is refunded (nothing was captured); the slot is released. If the scheduler is not running, this will not happen — confirm the scheduler's state before filing this as a defect.

7. **Reschedule.** Preconditions: Confirmed booking, no deposit owing, more than 24 hours before the appointment (default notice window). Steps: reschedule to a different open slot. Expected: appointment stays **Confirmed** at the new time; no money moves; no notification of any kind is sent (**Notifications** (Part 10), **the state-transition table** (Part 8)). Refusal variant: attempt to reschedule a booking that is still Pending Payment — expected refusal, "Only confirmed appointments can be rescheduled."

8. **Cancel — inside vs. outside the notice window.** Preconditions: Confirmed booking, ₹360 deposit already captured, nothing else paid. Steps A: cancel with more than 24 hours' notice. Expected A: full refund of **₹360**. Steps B: cancel inside the 24-hour window (or after the appointment time has passed). Expected B: the ₹360 deposit is fully forfeited; refund = **₹0** (there is nothing paid beyond the deposit to give back in this example).

9. **Double-booking attempt (one-to-one service).** Preconditions: a single-staff offering with exactly one open slot. Steps: submit two booking attempts for the same slot/staff at effectively the same time. Expected: exactly one succeeds; the other is refused at the database level — the customer sees the slot as unavailable, not a raw error.

10. **Capacity limit reached (class/venue service).** Preconditions: a capacity offering with, e.g., 10 seats, already fully booked for a slot. Steps: attempt an 11th booking on that slot. Expected: refused at the database level. The availability search should already have shown 0 remaining seats for that slot before the attempt — if the search still advertises open seats while the booking is refused, that mismatch is itself a real defect, since both numbers are supposed to be derived from the same live count.

11. **Plan-gated store.** Preconditions A: a store on a plan with Appointments explicitly disabled (Free/Starter). Steps A: attempt to open or call any Appointments surface. Expected A: refused, 403, "Appointments is not available in your current plan." Preconditions B: a store whose plan is missing its Appointments entitlement row entirely (a misconfiguration — should not exist on a healthy release). Steps B: same as A. Expected B: **allowed** — this is the fail-open case (**the plan-entitlement gate** (Part 11)); confirm before release that every plan tier actually has its row, since this case produces no error to catch it by.

Admin-side flow checklist

Every distinct admin flow in this module, for release/UAT test execution. Preconditions, exact control labels, and expected end state for each. Nothing here is assumed — flows not directly confirmable from source are marked `UNCONFIRMED`.

Setup flows

1. **Create your first offering.**
   - Preconditions: none.
   - Steps: Appointments → **Offerings** tab → **Add offering** → fill Name, Price, Duration (mins), and any other desired fields → click **Save**.
   - Expected end state: the offering appears in the Offerings list with status **Active**; the "Add your first service" item on the Settings checklist ticks off.

2. **Add a team member and give them working hours.**
   - Preconditions: if adding a Person-type resource, at least one free staff seat under your plan (or none, if your plan/addon has zero seats — see the seat-usage banner).
   - Steps: Appointments → **Team & Resources** → **Add** → choose Type **Person** → (either toggle **Link an existing user instead of inviting a new one** and enter the ID, or leave it off and fill Name/Email/password) → optionally toggle **Set an initial weekly shift** and pick a day + start/end time → **Save**. Then, from the new row's **Availability** icon, add at least one **Weekly shift** if none was seeded.
   - Expected end state: the resource appears in the Team & Resources list, Active; its Availability screen shows at least one weekly shift; it is now assignable to offerings and returns real time slots on the New appointment screen.

3. **Assign a team member to an offering.**
    - Preconditions: the offering and the team member both already exist.
    - Steps: Offerings → **Edit** the offering → **Team** section → select the team member(s) → **Save**.
    - Expected end state: the service now offers time slots against that team member on the Calendar / New appointment screen (assuming the team member also has a shift configured — see flow 2).

4. **Set booking rules and reminder cadence.**
    - Preconditions: none.
    - Steps: Appointments → **Settings** → fill in Minimum notice, Booking horizon, Slot length, Cancellation window, No-show fee %, and/or Default deposit, and/or Reminder cadence → **Save**.
    - Expected end state: values persist on reload; “Set booking rules” and “Set reminder cadence” checklist items tick off once a non-zero deposit/no-show value or at least one reminder time is saved, respectively. **Note for the tester:** no validation blocks a nonsensical value (e.g. negative minimum notice) — this is expected per the finding in Part 2, not a bug to file.

5. **Apply a preset that overwrites existing settings.**
    - Preconditions: at least one booking-rule value already saved (to observe the overwrite).
    - Steps: Settings → click a preset chip under **Start from a preset** → confirm in the **Apply this preset?** dialog.
    - Expected end state: all six booking-rule fields are replaced with the preset’s values; previously-saved custom values are gone with no way to undo.

6. **Build an intake (booking) form for a service.**
    - Preconditions: the offering has been saved at least once already.
    - Steps: Offerings → Edit the offering → **Manage intake questions** → **Add a question** → fill Question text, Answer type, required/optional, and (for choice types) at least one option → save the question. Repeat as needed; drag to reorder.
    - Expected end state: the question(s) appear in the live preview pane in the set order, and (per the customer-portal section of this guide) render on the actual booking form for that service.

Day-to-day flows

1. **Book a walk-in appointment.**
    - Preconditions: at least one Active offering with an assigned, available team member.
    - Steps: Calendar → floating walk-in button (or double-click an empty lane slot) → toggle **Add contact details** on → fill Contact name/phone → pick **Service**, **Team member** (or Anyone available), **Time** → **Save**/submit.
    - Expected end state: the booking appears on the calendar in the chosen resource's lane at the chosen time, with source "Booked in person" and status Awaiting payment or Confirmed depending on whether a deposit applies.
2. **Book an appointment on behalf of an existing customer (staff-booked, not walk-in).**
    - Preconditions: same as above; an existing customer record to search for.
    - Steps: Calendar → **New booking** button (or New appointment page) → toggle **Use an existing contact** on → search and select the customer → pick Service/Team member/Time → submit.
    - Expected end state: booking appears with source "Booked by staff."
3. **Record a cash/offline payment against a booking.**
    - Preconditions: a booking exists with an order/payment set up (or use **Collect payment** to start one) and an outstanding amount.
    - Steps: open the booking → Payment section → **Record payment** (or **Collect payment** if no order exists yet) → confirm/adjust the pre-filled amount → submit.
    - Expected end state: "Paid so far" increases by the recorded amount; if it reaches the full total, the panel shows "There's nothing due on this booking right now."
4. **Send a payment link to a customer.**
    - Preconditions: a booking with an outstanding deposit or balance.
    - Steps: open the booking → Payment section → **Send payment link** → confirm the amount shown (deposit or balance) → choose a **Send via** channel (Copy link / Email / WhatsApp if connected) → set **Link expires in** (≥1 day) → send.
    - Expected end state: link status shows "Awaiting payment" on the panel; once paid, status updates to "Paid via payment link."

5. **Check a customer in, start, and complete a booking.**
    - Preconditions: a Confirmed booking, on the day of the appointment.
    - Steps: open the booking → **Check in** → (when service begins) **Start** → (when finished) **Complete**.
    - Expected end state: status progresses Confirmed → Checked in → In progress → Completed, with a "Checked in at" timestamp recorded and each transition logged in Activity.
6. **Run My Day (staff self-service view).**
    - Preconditions: logged in as a staff user linked to a Team & Resources entry.
    - Steps: log in → land automatically on the simplified My Day view (no Offerings/ Team & Resources/Settings tabs visible).
    - Expected end state: only that staff member's own bookings for the day are shown, actionable the same way as the full detail panel. `UNCONFIRMED` — the exact card layout and fields of My Day, beyond the parts reviewed for this pass, were not exhaustively re-verified against every possible booking state; treat the core check-in/start/complete actions as confirmed, but verify any additional My Day-specific field visually during the UAT walk.

Exception flows

1. **Cancel a booking.**
    - Preconditions: a booking not already in an end state (Completed/Cancelled/No-show).
    - Steps: open the booking → **Cancel** → optionally enter a reason → confirm in the dialog.
    - Expected end state: status becomes Cancelled, the slot frees up on the calendar, no further action buttons appear.
2. **Reschedule a confirmed booking.**
    - Preconditions: booking status is exactly Confirmed (Reschedule is not offered from any other status).
    - Steps: open the booking → **Reschedule** → pick a new date and, optionally, a different team member → pick a new time slot (the booking's own current slot is also offered) → confirm.
    - Expected end state: booking moves to the new slot, old slot frees up, status remains Confirmed. If the new slot gets taken by someone else first, the dialog stays open and shows an error rather than closing.

3. **Mark a booking as no-show.**
    - Preconditions: booking status is Confirmed.
    - Steps: open the booking → **No-show**.
    - Expected end state: status becomes No-show; if a no-show fee is configured, it reflects in that day's money summary bar.
4. **Deactivate an offering or a team member/resource.**
    - Preconditions: the offering/resource exists.
    - Steps: from the respective list → **Deactivate** → confirm.
    - Expected end state: it disappears from the Active-only list view and stops being bookable going forward; existing/past bookings are unaffected; for a team member, their staff login to the rest of the admin panel is untouched.
5. **Re-activate a deactivated offering.** `UNCONFIRMED` — the deactivate confirmation copy states this is reversible ("This can be reversed by re-activating it later"), but the specific control to do so was not directly located/read in this session (most likely a status toggle on the same Edit form, reached via **Active only** switched off then **Edit**). Confirm the exact control during the UAT walk before relying on this as written.
6. **Cancel an active payment link.**
    - Preconditions: a payment link exists in "Awaiting payment" status.
    - Steps: open the booking → Payment section → cancel the active link → confirm.
    - Expected end state: link status becomes "Link cancelled"; **Send payment link** becomes available again for a fresh link.

Customer-side flow checklist

Each flow below is written as a literal script: preconditions, the exact button/field labels the customer sees, and the expected end state (including the exact amount at every money step). Anything not directly confirmed in source is marked `UNCONFIRMED` rather than asserted.

1. Book as a guest, no deposit required - Preconditions: signed out; chosen service has no deposit policy (or deposit is switched off). - Steps: Open `/book` → tap a service ("Select") → (if asked) choose staff or "Any available" → tap a date, then a time slot (hold starts, 5:00 countdown) → fill required questions + name, phone, and link channel (WhatsApp default) → tap "Continue" → review screen shows "No payment is required now — you'll pay at the time of your appointment." → tap "Confirm booking". - Expected end state: confirmation screen,

title “Booking confirmed”, heading “You’re all set!”; a “Save your booking link” box with a Copy button. Status is **Confirmed**. No amount is shown as due anywhere in this flow.

2. Book signed in, no deposit required - Preconditions: signed in; chosen service has no deposit policy. - Steps: same as above, minus the contact/channel fields (identity shown read-only) and minus the save-link box at the end (signed-in customers instead get “View my bookings” / “Book another”). - Expected end state: confirmation screen, “Booking confirmed” / “You’re all set!” Status **Confirmed**.

3. Pay a deposit, signed in (inline) - Preconditions: signed in; chosen service’s deposit policy is not “none”. - Steps: same booking steps as above through Confirm → review screen shows the deposit notice (“A deposit is required to confirm this appointment…”) → tap “Confirm booking” → lands on the payment screen, title “Pay deposit”, heading “Reserve your booking”, amount labelled “Deposit due” showing the server-fetched figure → tap “Pay deposit” → complete the Razorpay payment. - Expected end state on success: confirmation screen flips to “Booking confirmed” / “You’re all set!”; status **Confirmed**. On decline/dismiss: booking stays **Pending payment**, Pay button available again, amount unchanged (re-fetched, not stale).

4. Book as a guest with a deposit owed - Preconditions: signed out; chosen service’s deposit policy is not “none”. - Steps: same as flow 1 through Confirm → review screen shows the SAME deposit notice text as flow 3 (not the no-payment notice) → tap “Confirm booking” → guest is routed straight to the confirmation screen (never the inline payment screen). - Expected end state: confirmation screen, title “Booking pending payment”, heading “Almost there!”, subheading “Pay your deposit to confirm this appointment.” Save-box heading “Pay your deposit to confirm” with a copyable link. Status is **Pending payment** — never shown as confirmed at this point.

5. Pay a deposit via the guest link (after flow 4) - Preconditions: a guest booking sitting at **Pending payment**, holding its manage link. - Steps: open the saved `/book/manage/:token` link → detail card shows status “Pending payment” and a “Still owed” row → tap “Pay deposit” → lands on the same payment screen as flow 3 (amount fetched fresh) → complete payment. - Expected end state on success: booking status flips to **Confirmed**; returning to the manage link shows “Confirmed” and the money block now shows the deposit amount under “Paid”.

6. Pay an outstanding balance (My Bookings or guest link) - Preconditions: a booking whose deposit is paid (status **Confirmed** or later) but with `Still owed > 0`. - Steps: from My Bookings list or detail (signed in) or the manage link (guest), tap “Pay balance” → lands on the payment screen, title “Pay outstanding balance”, heading “Complete your payment”, amount labelled “Balance due” → complete payment. - Expected end state: “Still owed” drops to reflect the new payment; “Paid” increases by the same amount.

7. Abandon a payment and return later - Preconditions: any of flows 3/5/6, mid-payment. - Steps: open the Pay button, then close the Razorpay window (or the tab) without paying →

return later via the same route (confirmation link, My Bookings, or manage link). - Expected end state: booking status unchanged from before the attempt (still **Pending payment** if it was a deposit); Pay button and amount both present and correct on return — the app re-fetches the amount, it does not remember a stale figure or show any penalty for having walked away.

8. Try to pay a leg that's already settled - Preconditions: a booking with nothing currently owed on the leg being requested (e.g. a stale "pay balance" link reused after already paying). - Steps: open the pay link/CTA. - Expected end state: no Pay button, no error message — the customer is redirected onward automatically (to Confirmed, My Bookings, or the manage page depending on entry point).

9. Reschedule (signed in or guest) - Preconditions: booking within the merchant's reschedule window, not cancelled/completed/no-show/in-progress, and not awaiting payment. - Steps: open the booking detail (My Bookings or manage link) → tap "Change time" → pick a new date/time (same staff/resource only) → tap "Confirm new time". - Expected end state: dialog closes, booking detail shows the new date/time, status unchanged. If the exact new slot gets taken first: "That time was just taken. Please pick another." — dialog stays open with fresh times.

10. Cancel (signed in or guest) - Preconditions: booking within the merchant's cancellation window, not already cancelled/completed/no-show, and not awaiting payment. - Steps: open the booking detail → tap "Cancel booking" → confirm dialog appears ("Cancel this booking?") → tap "Yes, cancel booking". - Expected end state: status flips to `cancelled`; any refund follows the merchant's policy and is reflected in the money block (Refunded / refund-in-progress / refund-failed messaging as applicable).

11. View bookings (signed in) - Preconditions: signed in, has at least one booking, store has appointments enabled. - Steps: My Orders → Bookings tab → list shows Upcoming/Past. - Expected end state: each row shows status, and (if applicable) "Still owed" + a pay CTA. Tapping a row opens the full detail.

12. Use the guest manage link - Preconditions: a guest booking's manage link. - Steps: open `/book/manage/:token`. - Expected end state: same detail view, money block, and Cancel/Reschedule/Pay actions as the signed-in detail page — reached without any account or login. An invalid/garbled/unrecognised token always shows the same generic "This booking link isn't valid" message (never reveals whether the token was almost-right). Whether a *previously valid* token can go invalid purely by elapsed time is `UNCONFIRMED` in this source — no expiry field or check was found on the manage token itself.

Part 13 — What Isn’t Finished Yet

In the admin panel

A short, honest list of things merchants sometimes look for that aren’t there today:

- **The time limit for an unpaid booking is enforced but was not editable from Settings.** A booking awaiting its deposit is released automatically once a store-level time limit passes, and that limit is applied on every sweep — but the Appointments Settings form never offered it, so you could not see or change it. It is being added as a field (5 minutes to 7 days); until it appears, the platform default applies. See **Appendix B**.
- **No visible “spots remaining” count on a group/capacity booking’s calendar chip.** The system enforces the capacity limit when offering slots, but doesn’t show you the live headcount on the calendar itself.
- **No exclude/block-slot control directly on the calendar.** Blocking time off is done entirely through a resource’s Availability screen (date overrides & blocks), not by clicking a slot on the day grid.
- **Intake answers are not lost — only un-rolled-up.** An earlier edition of this guide implied submitted answers were unreadable because the builder’s “Submissions” panel is a stub. That is wrong: every answer appears on the individual booking’s detail panel, formatted for all eleven question types, with a proper empty state. What is genuinely missing is a single screen listing all submissions for one form at once. See **Appendix B**.
- **The Activity timeline only logs events that genuinely happened** (booked, checked in, started, completed, cancelled, no-show). It does not show “deposit captured,” “reminder sent,” or “WhatsApp confirmation sent” — those aren’t tracked as logged events in this system yet, even though reminders/payment links do get sent.
- **No separate GST-itemised invoice document for a booking** — GST rate and SAC code are stored on the service for tax purposes, but there is no invoice download visible from the booking detail panel.
- **SMS payment links are not available** on any plan — the option is always shown as disabled. The reason is that no SMS provider is connected to the platform at all, for any feature; the option stays off until one is, rather than promising a message nothing could send.
- **Booking a service currently requires assigning at least one team member to it, and any newly added team member/resource needs a working-hours shift set up before they can be booked at all** — a service with nobody assigned, or a brand-new resource with no shift, simply won’t offer any time slots.

What is not finished (current limitations, stated plainly)

- **Guest phone/OTP verification is not yet a hard requirement to book.** A guest can currently complete a booking without ever successfully verifying the code sent to their phone. A stricter gate is a known, separate, not-yet-shipped decision.
- **No SMS channel.** Every notification is WhatsApp and/or email only.
- **WhatsApp "sent" only means "accepted by the send pipeline," not "delivered or read."** There is no delivery-confirmation loop visible in this booking system.
- **Rounding of fractional-paisa deposit amounts is confirmed sound.** It was previously listed as unconfirmed. A deposit rounds to the nearest paisa using a float-safe round-half-up, applied at the moment the amount is first calculated — not re-rounded later. See **Appendix B**.
- **No notification exists for check-in, start-of-service, completion, or reschedule.** This is confirmed, not a gap — see **Notifications** (Part 10) and **Known limitations and open defects** (Part 13). A customer who reschedules gets no automatic confirmation of the new time beyond what's shown on-screen in the moment.
- **The refund wording is confirmed correct, not a gap.** The customer sees three distinct states — refund in progress, "Refunded" with the amount, and a failed state asking them to contact you. See **Appendix B**.

Known limitations and open defects (release audience)

Only items directly confirmed from source this session are listed. Each is one line of symptom, one of impact, and an explicit release-blocking call.

1. **Guest phone verification (OTP) is not enforced.** Impact: a guest can complete and pay for a booking with an unverified phone number, so the number a booking is tied to is not guaranteed to belong to the person who booked it. **Not a release blocker** — this is a stated, deliberate, already-shipped interim state pending a separate future decision, not a regression.
2. **No notification exists for check-in, start-of-service, completion, or reschedule.** Impact: a customer who is rescheduled gets no automatic message of the new time; a merchant relying on notifications to signal "your appointment is starting/done" has no such signal today. **Not a release blocker** — this is existing, long-standing behaviour (the notification system simply has no event type for these four cases), not a new regression, but it is worth flagging to product as a real gap.

3. **A WhatsApp “sent” result only means the message was accepted by the send pipeline, not that it was delivered or read.** Impact: no delivery confirmation loop exists for any channel. **Not a release blocker** — an industry-standard limitation of the integration, not a bug.
4. **The Appointments entitlement gate fails open if its plan_limits row is ever missing**, rather than erroring (**the plan-entitlement gate** (Part 11)). Impact: a misconfigured or incompletely-migrated plan could silently grant full Appointments access to a plan that should not have it, with no error surfaced anywhere. **Release-blocking IF AND ONLY IF a release’s migrations do not include an explicit row for every plan tier** — this is a pre-flight check to run every release, not a currently-known-broken state.
5. **The scheduling component that runs all three background sweeps (**Timer and background-job behaviour** (Part 8)) currently runs as a single instance with no leader election.** Impact: theoretical double-fire risk if ever scaled to multiple instances without also adding leader election. **Not a release blocker today** — the sweeps’ own row-locking already prevents a double-fire from doing any double harm, and this is a known, documented constraint rather than an active defect.

Part 14 — When Things Don’t Look Right

Every row below is a symptom someone actually reports, paired with what causes it and what to do. Work top to bottom: the first four rows account for the large majority of “Appointments isn’t working” reports, and none of them is a fault in the software.

No times are being offered

This is the single most common problem, and the storefront gives you the same message — *“No open slots for this selection”* (or “no one available”) — for several different causes. Check them in this order:

#	Check	Where	Why it does this
1	Does the person have at least one **Weekly shift**?	Appointments → Team & Resources → open the entry → Availability	A resource with zero shifts is treated as never available. No slot is ever offered for them, anywhere. There is no warning about this at booking time — the only hint is the reminder text on the availability editor itself.
2	Does the offering have anyone ticked under **Team**?	Appointments → Offerings → open the service → Team	A service with nobody assigned has nobody to perform it, so it offers no slots even though it exists and is priced.
3	Is **Visible for online booking** switched on, and is the offering **Active**?	Appointments → Offerings → open the service	If it's off, the service is staff-booking only — it won't appear in your storefront's service list at all. An inactive offering is not bookable either.
4	Are you asking for a time inside your own **Minimum notice** window?	Appointments → Settings	Minimum notice pushes the earliest bookable slot forward. If your notice is set to, say, 24 hours, today has no slots by design.
5	Are you looking beyond your **Booking horizon**?	Appointments → Settings	The horizon caps how far ahead customers can book. Dates past it legitimately show nothing.
6			

#	Check	Where	Why it does this
	Is the slot already taken?	Appointments → Calendar, on that day	For a one-to-one resource, a booked slot is gone; for a capacity/group service, the slot disappears only once it's full.

Other symptoms

Symptom	What's actually happening	What to do
Appointments shows an upgrade message instead of the calendar	Your subscription doesn't include Appointments. The module is plan-gated at the moment each screen loads.	Upgrade, or check your current plan's entitlements on your billing screen. See **Part 11**.
A booking sits at "Pending payment" and the customer swears they booked	They did. A booking whose deposit is greater than zero is created as **Pending payment** and only becomes **Confirmed** once that deposit is actually paid.	Send them the payment link from the booking's **Payment** section, or record the payment if you took it another way. See **Part 9**.
A "Pending payment" booking vanished on its own	This is intended. Once your **deposit timeout** (default 30 minutes) passes with the deposit unpaid, an automatic sweep cancels the booking and frees the slot.	Nothing to fix. If customers need longer to pay, raise the deposit timeout in Settings.
The customer says the booking "expired" while they were still filling in the form	Picking a slot creates a 5-minute **hold** while they finish. If they take longer, the hold lapses and they're asked to pick a time again.	Nothing to fix. This is a separate, much shorter timer than the deposit timeout — don't confuse the two. See **Part 8**.
A staff member only sees "My Day" and can't find Offerings or Settings	Working as designed. A team member who is linked to a bookable resource lands on a simplified My Day view and only sees **Calendar** and **My availability**.	Use an owner/admin login for setup work. See **Part 11**.
The "Manage intake questions" button is greyed out	The intake form lives on a saved offering. The button stays disabled, with the tooltip *"Save this offering first to add intake questions,"* until the offering has been saved once.	Save the offering, reopen it, then add questions.
You can't add another team member	You're at your plan's staff-seat limit. Most plans include zero staff seats by default — seats	Use the **Buy seat** link on the seat-usage banner. See **Part 11**.

Symptom	What's actually happening	What to do
	are generally bought as an add-on.	
Saving an 11th (or 21st…) service shows a limit message	Your plan caps how many active offerings you can have. The message appears inline on the form, not as a pop-up, so it's easy to miss.	Deactivate a service you no longer sell, or add capacity.
A customer's Cancel button is greyed out	They're inside your **cancellation window** — the deadline you set in Settings. The reason is shown next to the button.	Cancel it for them from the admin panel; staff cancellation has no notice-window restriction at all.
A customer can't reschedule	Two legitimate blocks: a deposit is still outstanding (pay or cancel first), or the booking is already **Checked in/In progress**. Cancel is still allowed on a checked-in booking; only reschedule is blocked.	Reschedule it for them from the booking detail panel.
Nobody got a message when a booking was rescheduled, checked in, or completed	No such notification exists. The system sends exactly seven kinds of message: booking confirmed, reminder, cancelled, no-show, a new-booking alert to you, the OTP code, and a payment link.	Tell the customer the new time in the moment. See **Part 10** and **Part 13**.
Reminders aren't arriving by SMS	There is no SMS channel. Every notification is WhatsApp and/or email.	See **Part 13**.
WhatsApp says "sent" but the customer says they never saw it	"Sent" means the send pipeline accepted the message, not that it was delivered or read — there's no delivery-confirmation loop back into the booking system.	Follow up by phone for anything time-critical.

Symptom	What's actually happening	What to do
The times on the calendar don't match what you expected	All availability maths — open hours, the slot grid, minimum-notice cutoffs — runs in **your store's configured timezone**, not the server's and not the viewer's browser.	Confirm your store timezone is right. See **Part 10**.
A nonsensical booking rule saved without complaint	Known gap: the six numeric booking-rule fields in Settings have no validation. A negative minimum notice or a 500% no-show fee will save silently.	Re-read your Settings values carefully; nothing will warn you. See **Part 2**.
The Activity timeline doesn't mention the deposit or the reminders	By design. Activity shows only events that genuinely occurred as tracked steps — booked, checked in, started, completed, cancelled, no-show. Deposit captures and reminder sends aren't logged as timeline events.	Use the **Payment** section for money history. See **Part 6**.
There's no invoice to download for a booking	Not built yet. An offering's GST rate and SAC code are captured on the service, but the booking detail panel shows no separate GST breakdown and generates no invoice document.	See **Part 13**.
The Submissions panel on the intake-form screen is empty	That roll-up view is still a stub showing a "coming soon" note — nothing has been lost.	Open any booking and read its answers on the booking's own detail panel. A roll-up screen is on the fix list.

Part 15 — Quick Reference — Screen Index

Every screen in the Appointments module, where it lives, and which part of this guide covers it in full.

Admin panel — your dashboard

The admin panel is a desktop tool. Four tabs run across the top of every Appointments screen: **Calendar**, **Offerings**, **Team & Resources**, **Settings**.

Screen	How you get there	What it's for	Covered in
Calendar (day / week / month)	Sidebar → **Appointments**	The day-to-day booking board, with a money summary bar for the day	Part 5
My Day	Sidebar → **Appointments**, when you're signed in as a team member linked to a resource	A simplified personal schedule instead of the full calendar	Parts 5, 11
New appointment	Calendar → **New appointment**	Book on a customer's behalf, in person or over the phone	Part 5
Booking detail panel	Click any booking on the Calendar	Everything about one booking: details, intake answers, activity timeline, payment, and every action	Part 6
Reschedule dialog	Booking detail panel → **Reschedule**	Move a booking to a different time with the same resource	Part 6
Payment section	Inside the booking detail panel	Totals, deposit, balance due, send a payment link, record a payment	Part 9
Offerings list	**Appointments → Offerings**	Every service you sell, active and inactive	Part 3
Offering form	Offerings → **New offering**, or click a service	Name, price, duration, deposit, capacity, GST, and which team members can deliver it	Part 3
Intake question builder	Offering form → **Manage intake questions** (available	The questions customers answer	Part 3

Screen	How you get there	What it's for	Covered in
	only after the offering has been saved once)	when booking this service	
Team & Resources list	**Appointments → Team & Resources**	Your bookable people, rooms and equipment, plus the staff-seat usage banner	Part 4
Add/edit team member or resource	Team & Resources → **Add**, or click an entry	Create a person (optionally with a login) or a non-person resource	Part 4
Availability editor	Open a Team & Resources entry → Availability	Weekly shifts, and date-specific exceptions	Part 4
My availability	Sidebar → **Appointments**, as a staff member	The same availability editor, scoped automatically to your own resource	Parts 4, 11
Settings	**Appointments → Settings**	The first-run checklist, business-type presets, and all six numeric booking rules plus reminder cadence	Part 2

Your storefront — what the customer sees

The storefront is responsive: everything below works on a phone as well as a desktop.

Screen	How a customer gets there	What happens there	Covered in
Booking page	`yourstore.com/book` — your homepage if the store is in booking mode, otherwise any link you place	The list of services that are visible for online booking	Part 7
Staff / resource picker	After choosing a service, when more than one person or resource can deliver it	Pick a specific person, or “Any available”	Part 7
Time picker	After choosing a service (and a person, if asked)	Pick a date and a time; picking one places a 5-minute hold	Parts 7, 8
Details and intake form	After picking a time	Contact details, plus whatever intake questions you configured	Part 7
Review and confirm	After the details step	A summary of what’s being booked, and whether a deposit is owed	Part 7
Pay deposit / Pay outstanding balance	Straight after confirming, for a signed-in customer who owes a deposit; or any time afterwards from a booking’s own page	Pays the amount the server says is currently owed — never a number stored on the device	Parts 7, 9
Booking confirmed / Booking pending payment	After confirming	Two different screens: a booking with a deposit still owed is never shown as confirmed	Parts 7, 8

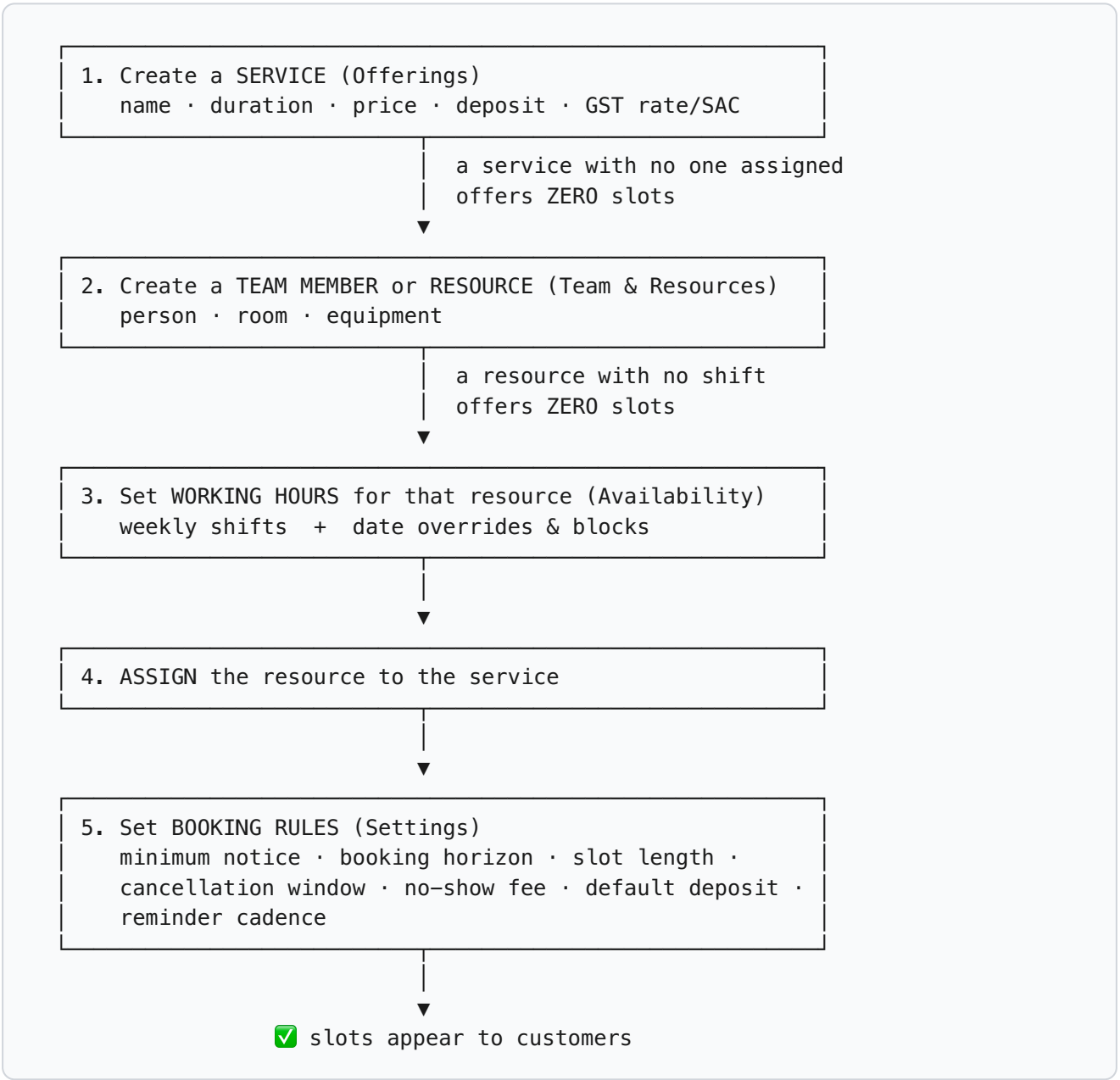
Screen	How a customer gets there	What happens there	Covered in
My Bookings	**My Orders → Bookings** (appears only once your store has services turned on)	Upcoming and past bookings, with amounts still owed and a link to pay	Part 7
Booking detail page	Tap any booking in My Bookings	The full booking, with cancel and reschedule where they're allowed	Part 7
Guest manage link	The personal link a guest is given when they book (`/book/manage/…`)	The same detail view and the same actions as a signed-in customer, with no account needed	Part 7

Part 16 — The Whole Module in Diagrams

Nine diagrams that between them cover every flow in Appointments. Each one says which Part explains it in words, so use this as the map and the Parts as the manual. Nothing here is new information — it is the same audited behaviour, drawn.

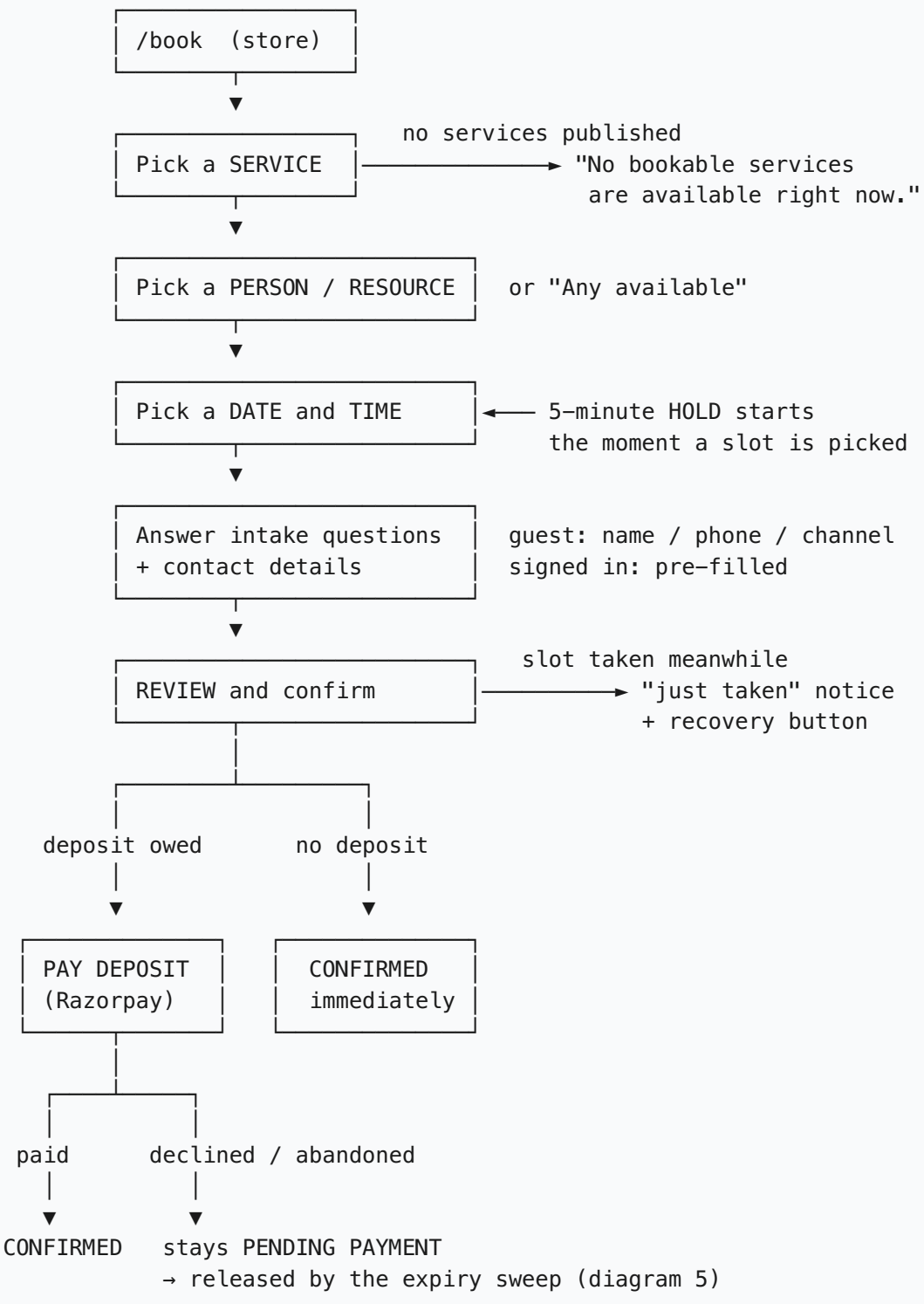
1. Setting up, in the order that actually works

Each step depends on the one above it. Skip a step and the symptom is always the same: **the customer sees no available times, with no error explaining why.** That silence is the single most common support case in this module.



Explained in words in **Part 2** (checklist), **Part 3** (services), **Part 4** (team and resources).

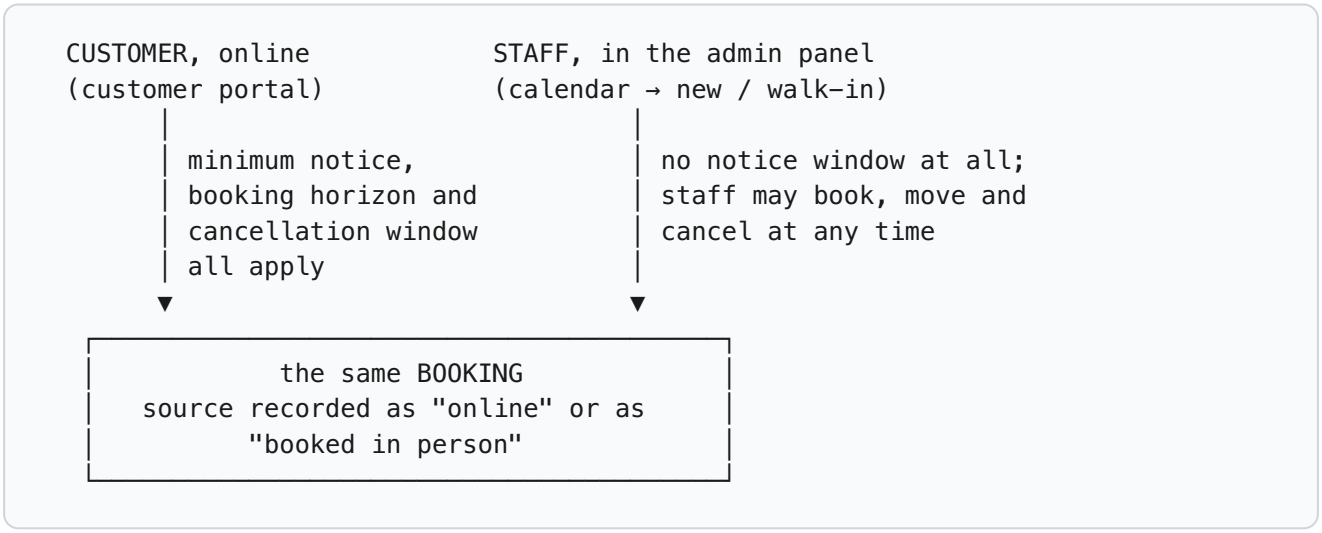
2. The customer’s journey, with every branch



Explained in words in **Part 7**. The deposit rules are **Part 9**.

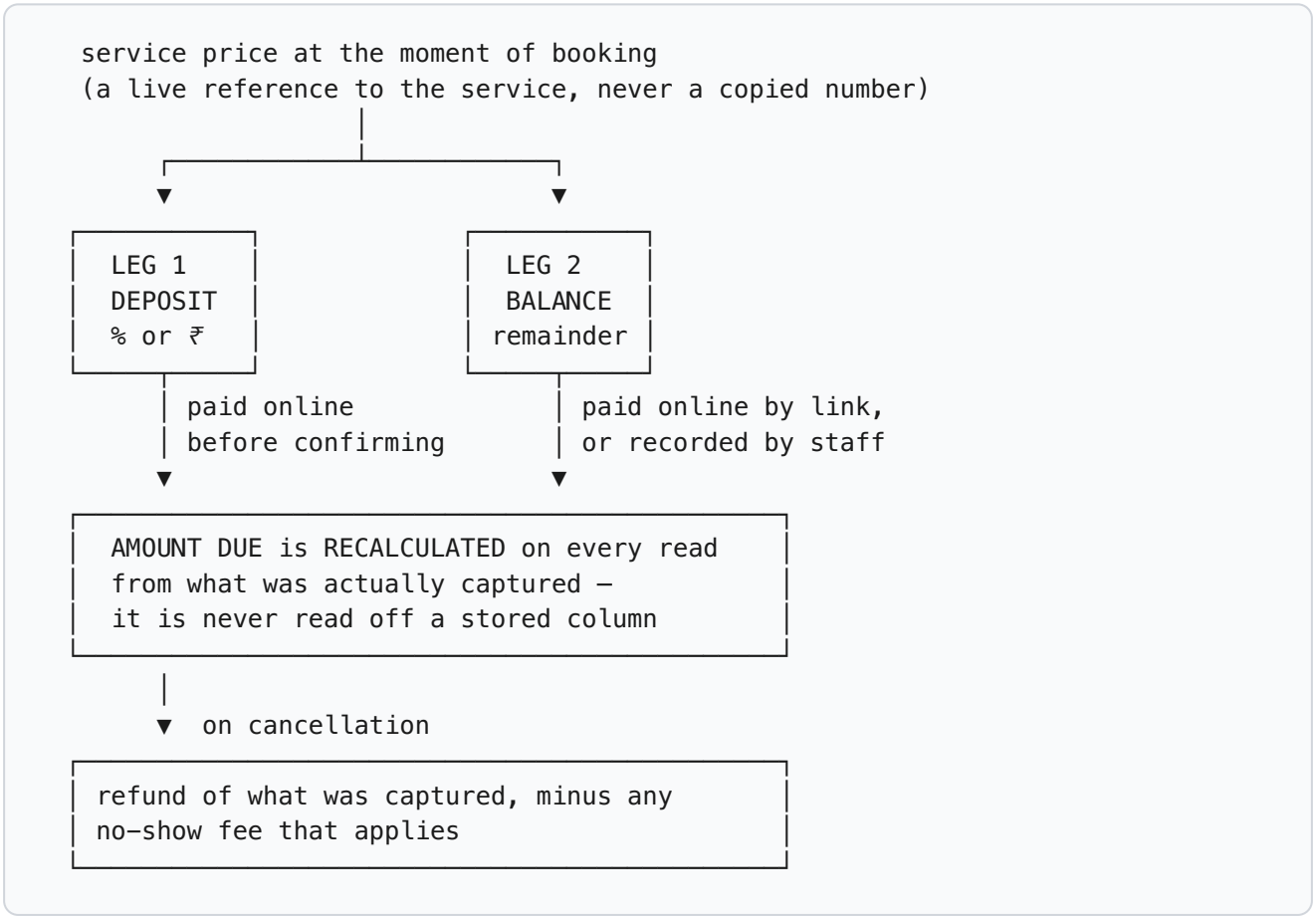
3. Where a booking can come from

All three paths land in the same booking, the same calendar and the same detail panel — only the recorded source and the notice rules differ.



Explained in words in **Part 5** (calendar) and **Part 8** (rules).

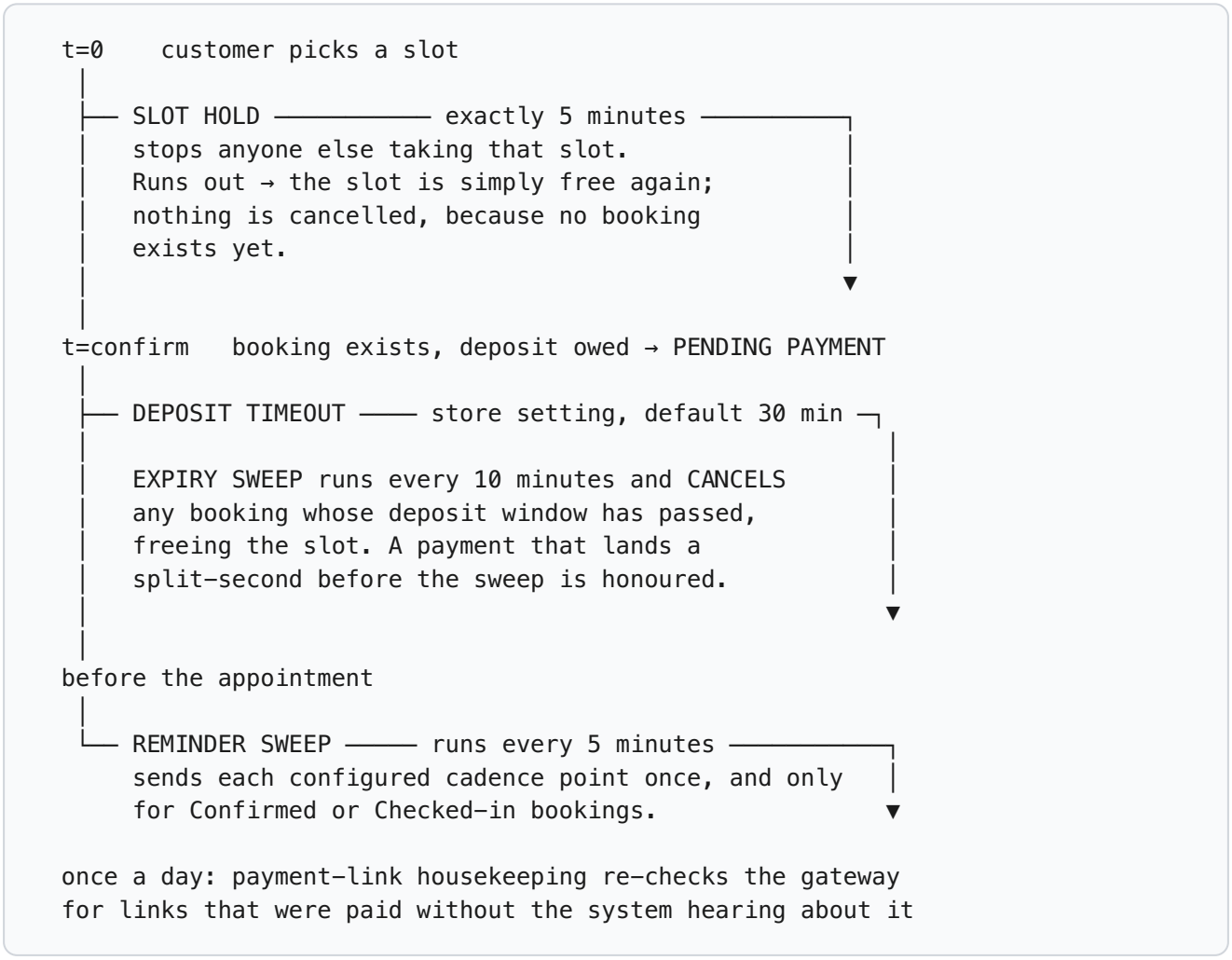
4. Money — two legs, never one lump sum



Explained in words in **Part 9**. The golden rule — money is never a stored number — is why a refund can never leave a stale “due” figure behind.

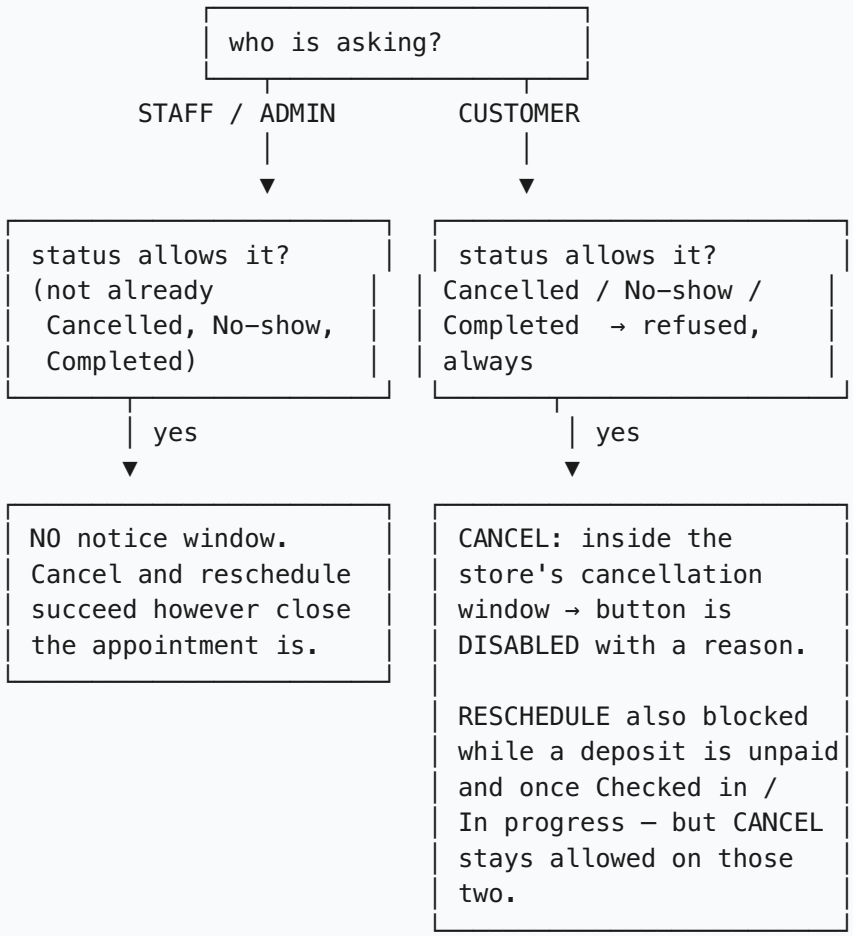
5. The three clocks

Three independent timers. Confusing them is the most common source of “why did my booking disappear?” questions.



Explained in words in **Part 8** (holds and sweeps) and **Part 10** (reminders).

6. Can this booking be changed? — the decision tree



The asymmetry is deliberate: only customers face a notice window. Explained in words in **Part 8**; the customer-side buttons are **Part 7**.

7. What sends a message — and what does not

There are exactly seven kinds of outbound message. There is no eighth.

EVENT	SENDS?	CHANNEL
booking confirmed	yes	WhatsApp
reminder (each cadence point)	yes	WhatsApp
cancelled	yes	WhatsApp
no-show	yes	WhatsApp
new-booking alert to merchant	yes	WhatsApp
one-time verification code	yes	WhatsApp
payment link	yes	WhatsApp / email / both
checked in	NO	— no such message exists
service started	NO	— no such message exists
completed	NO	— no such message exists
RESCHEDULED	NO	— no such message exists

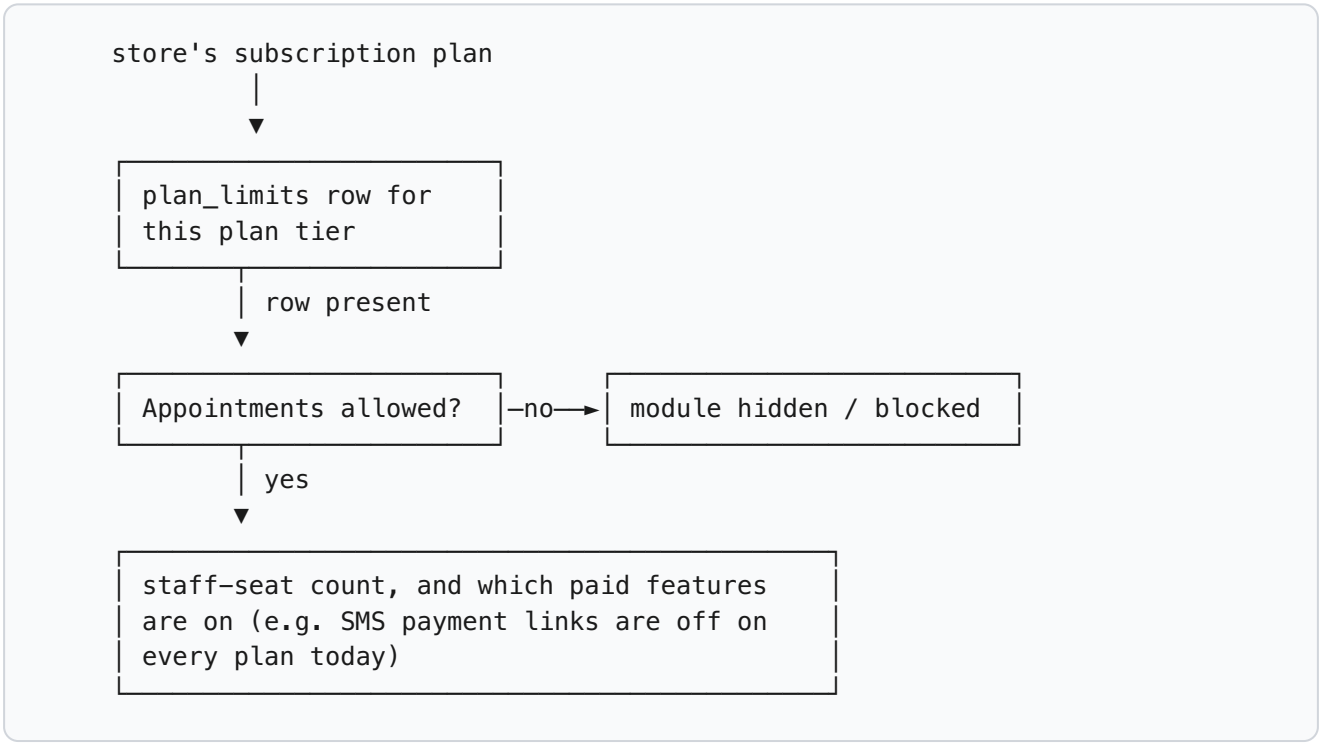
The bottom four are not failures to send — no message of that kind exists in the system. A rescheduled customer is told the new time on screen and nowhere else. Explained in words in **Part 10**, listed as a gap in **Part 13**.

8. Who can see and do what

OWNER / ADMIN	STAFF linked to a resource
Calendar (all resources)	"My Day" only (own bookings)
Offerings	— not visible
Team & Resources	— not visible
Settings	— not visible
—	"My availability" (own hours)

A staff login that is not yet linked to a resource sees a prompt to ask an admin to link it. Explained in words in **Part 1** and **Part 11**.

9. How a plan decides what you get



⚠️ **Release pre-flight, every release:** if the `plan_limits` row for a tier is missing entirely, the gate currently **fails open** — it grants access rather than refusing it, and says nothing. Confirm every plan tier has an explicit row before shipping. Explained in **Part 11**, tracked as a fix in **Part 13**.

Appendix A — Release Notes, Test Environment and Verification Coverage

This part is written for whoever is signing this module off — a tester, a release manager, or a merchant who wants to know how thoroughly this was checked before it reached them. It does not repeat how the feature works; that is covered elsewhere in this document.

What Is In This Release

Appointments is built across three separate codebases that deploy independently: the booking engine and data storage, the store-owner admin screens, and the customer-facing booking pages. All three currently sit on their staging branch, which is what powers the UAT (pre-production) test environment. Below is what each contains, in plain terms, grouped by what a person actually sees or experiences — not by individual code commits.

Booking engine and data storage

- Appointments as a bookable product: services (“offerings”), staff or resource assignment, weekly availability rules, and slot generation that respects buffers, minimum notice, and existing bookings.
- Deposits and partial payments: a booking can require a deposit instead of full payment, the remaining balance is tracked as a separate amount owed, and a booking that still owes a deposit is never marked “Confirmed.”
- A local stand-in for the online payment processor so the money flow (charge → the right amount lands against the right leg of the booking) can be tested without a live payment gateway.
- A payment link a guest can be sent by SMS or shown directly, so someone without an account can pay an outstanding deposit or balance themselves.
- Self-service cancel and reschedule for the customer, and admin-side cancel with an optional reason.
- No-show handling: a booking with an uncollected deposit is automatically released after a timeout window, correcting an earlier issue where this happened too early on servers not set to Indian time.
- Correct tax handling on invoices generated from a booking, and a fix so an invoice total’s line items now add up to the printed subtotal.

- A per-plan cap on how many bookable services a store can create, and a cap on staff seats, both enforced the same way the platform enforces its other plan limits.
- Several fixes to stop specific error conditions returning a raw server error (500) instead of a proper “not allowed” or “not found” response — these are invisible to a merchant when things go right, but matter when something goes wrong.

Admin panel (the screens a store owner uses)

- A full appointments section: Calendar (day/week/month), a “My Day” summary, Services, Team/Resources, Availability, and Settings.
- Calendar usability fixes: a “now” button that jumps to today, stacking of overlapping bookings, a wider time window when a booking falls outside normal hours, and a single consistent status legend.
- A day summary tile showing money expected and money still outstanding for that day.
- Recording a payment against a booking now defaults to the actual amount still owed, not the full order total or a stale balance figure.
- The payment-status label shown to staff now reads as a plain word (“Deposit paid”, “Awaiting payment”) instead of a raw internal code.
- A reschedule dialog for confirmed appointments, with the booking’s own current slot kept selectable.
- A booking-page design editor, so a store owner can lay out and style the public booking page the same way they style other storefront pages.
- A staff-seat usage indicator with a direct link to buy more seats when the limit is hit.
- A guard so a limited “staff” login role cannot reach the management screens meant for owners/admins.

Customer portal (what a shopper or guest sees)

- The public booking funnel: choose a service, pick a staff member or resource if applicable, pick a time slot, enter details, pay if required, and get a confirmation.
- “My Bookings” for a signed-in customer, plus the guest-facing “manage my booking” link for someone who booked without an account.
- Clearer payment-step messaging: the button names the specific amount it is about to charge, and the confirmation step no longer tells a guest no payment is needed when one still is.
- A retry option on a failed payment step instead of a dead end.
- The booking pages now pick up the store’s own branding (colours, fonts) instead of a generic default look.

- A “pay outstanding balance” option surfaced from “My Bookings” for a confirmed booking that still has money owed.

What is explicitly not part of this list

Store-editor (page-builder) changes, storefront chrome/header/footer changes, and other in-flight platform work also live on the same staging branches quoted above. They are not appointments features and are out of scope for this document.

Deployed vs. staged — read this before testing

Codebase	Latest staged commit	What is actually running on the UAT admin/test URL
Admin panel	`e60c076a` (2026-09-12)	`8d57b060` (2026-09-11) — **96 commits behind**, confirmed by comparing the running site’s own version file against the staging branch history
Customer-facing booking pages	`0af47061` (2026-09-12)	`0af47061` — matches exactly
Booking engine / data storage	`46537553` (2026-09-12)	Cannot be read the same way — see NULL below

The admin panel currently running on the UAT admin URL is **96 commits behind** the tip of its staging branch. Counting only the appointments-labelled commits in that gap, **86 of them have not reached the deployed admin panel**, including the “My Day” money tile, the week/month calendar views, the reschedule dialog, the booking-page design editor, the staff-seat indicator, and both of the Record Payment / payment-status-label fixes described above. **A tester on the current UAT admin URL is testing an appointments admin experience that is materially older than what this document just described.** Whoever owns this release needs to trigger a fresh admin-panel deploy before a meaningful admin-side sign-off can happen.

NULL — backend build identity is unverifiable by this method. The booking-engine and data-storage services do not publish a version file the way the two front-ends do, so which commit is actually running behind the API cannot be confirmed by reading a URL. The only way to know what backend code is live is to exercise a specific behaviour change through the API and observe the result — which is exactly what the fixture script described below does for the booking money fix. That fix is verified working locally, but not yet confirmed on UAT for lack of a bookable UAT store — see the next section.

Environment Reference

Purpose	URL	Notes
Admin panel (UAT)	`https://admin.uat.billionbiz.in`	Reachable (HTTP 200). Currently serving an appointments build 96 commits behind staging — see above.
API gateway (UAT)	`https://api.gateway.uat.billionbiz.in`	Reachable; a bare request to `/` returns 404 as expected — it only answers on defined API paths. Every request must carry the storefront's `Origin` header; without it, requests are rejected before even reaching the appointments logic.
Storefront used for booking (UAT)	`https://qa-admin.uat.billionbiz.in`	This is the **only** UAT store whose subscription currently allows appointments at all — see below. Despite the "qa-admin" name, this host serves the customer-facing storefront (its build matches the customer-portal staging tip exactly), not an admin panel; do not confuse it with the actual admin panel URL above.

Which UAT store can actually be used to test appointments

Confirmed by direct, unauthenticated request against each store's public appointments listing endpoint, same day as this document:

Store	Public appointments API	Usable for testing?
`qa-admin`	`200 OK`, returns an empty offerings list	Only store whose subscription passes the appointments plan gate — but it currently has **zero published bookable services**, so the funnel cannot be walked end to end as-is
`roy-store`	`503 PLAN_GATE_UNAVAILABLE`	No — no active subscription recognises appointments on this store
`tamil`	`503 PLAN_GATE_UNAVAILABLE`	No — same reason
`judith-store`	`503 PLAN_GATE_UNAVAILABLE`	No — same reason

In other words: as of this document, **no UAT store can currently be walked through a real booking end to end** without first creating at least one bookable service on `qa-admin`. A prepared script exists — dry-run by default, and human-approved before it writes — to create exactly that: one staff resource, a weekly availability window, and one paid service with a 30% deposit — using the product's own admin API, no direct database writes. It requires a merchant-admin sign-in token obtained by hand (UAT sign-in is rate-limited, so this is deliberately not automated) and explicit confirmation before it writes anything.

NULL — whether this fixture has actually been run is not established by this document. This section confirms the store/subscription landscape the script's own comments describe is still accurate today; it does not confirm whether Phase A of that script has since been executed, i.e. whether `qa-admin` currently has a bookable service live. Check the store directly before assuming either way.

Automated Test Coverage By Flow

Coverage below counts test files and individual test cases (`it` / `test` blocks) found in the source of each staging branch, and confirms — separately — whether the runner that actually executes on this project's CI actually picks each file up. A file that exists but isn't wired into a runner provides no real protection; both are reported.

Flow	Backend coverage	Admin panel coverage	Customer portal coverage	Automated or manual-only
Booking funnel (choose service → resource → slot → details → confirm)	Slot/availability engine, hold-and-confirm, intake-answer handling — 9 files	Screen-level regression specs for the admin booking screens — 3 files	Step-by-step funnel specs incl. accessibility — 10 files	Automated
Deposit & payment (quote, partial payment, guest pay link, record payment)	Money serialization, outstanding-balance math, deposit policy, deposit timeout, payment-link/manage-link, deposit auto-release — 10 files	Collect/record-payment and payment-link specs — 2 files	Payment step + local payment-simulator specs — 3 files	Automated
Reschedule & cancel (customer self-service and admin)	Hold-confirm, refund, no-show forfeiture, self-service eligibility — 6 files	Reschedule dialog spec — 1 file	Self-service and “My Bookings” specs — 2 files	Automated
Admin calendar & daily operations (day/week/month view, “My Day” money tile)	Day-summary service — 1 file	Calendar/week-month/day-summary/deep-link specs — 6 files	—	Automated (admin side); no customer-facing equivalent
Store setup (services, staff/resources, availability rules, staff-seat limits)	Offerings/resources/availability-rules/staff-seat-limit services — 9 files	Forms, seat-indicator, exclusion-on-reschedule specs — 3 files	—	Automated
Notifications & reminders	Notification service,	—	—	Automated, backend only —

Flow	Backend coverage	Admin panel coverage	Customer portal coverage	Automated or manual-only
(booking confirmation, reminder sweep, OTP)	reminder-sweep job, OTP service, realtime socket event — 4 files			**no test confirms a real SMS/ notification is actually sent and received; only that the service attempts to queue one**
Plan/feature gating (appointments hidden or blocked for stores without the plan)	Feature-gate route specs — 2 files	Nav-and-gate, booking-editor-gate specs — 2 files	—	Automated
Booking-page design editor (admin) / live rendering (customer portal)	—	Step-format panel, seed-defaults, feature-gate specs — 3 files	Preview-bridge, template-CSS specs — 4 files	Automated
Cross-service, end-to-end flow (a real booking placed through the admin API, paid, and reconciled against an order/ invoice)	A hand-written API-level suite (2 scripts, ~11 checks) exists and is registered to run	—	—	**Written but not yet merged to this test repo's staging branch — not currently part of any run a tester would trigger.** Today, this flow is verified only by manually walking the product, or by the one-off UAT fixture script referenced above.

Flow	Backend coverage	Admin panel coverage	Customer portal coverage	Automated or manual-only
Full end-to-end browser walk (Playwright, simulating a real user session)	—	—	—	**None exists.** No Playwright spec for appointments/ booking was found anywhere in this project's automated end-to-end suite.

Raw counts behind the table above: **118 spec files** across the three codebases (57 backend, 33 admin panel, 28 customer portal), containing **856 individual test cases** (397 backend, 314 admin panel, 145 customer portal) — counted by reading file names and counting the individual test declarations inside each file's current staging-branch content, not by running anything.

A wiring gap worth knowing before you trust a "tests passed" claim

The admin-panel and customer-portal spec files above are confirmed wired into this project's actual test runners — both runners' file-discovery pattern was read directly and explicitly includes the folder these specs live in, so a run through the standard process will pick every one of them up.

The backend picture is split in two. All 57 backend appointments spec files live alongside the code they test and are picked up by that service's own `test:src` command — this works today, on the current staging branch. But the backend project's **other** test entry point — its plain `test` command — delegates to a separate, hand-written suite that lives in this project's shared testing repository and does not scan the service's source tree at all; that separate suite's own configuration currently only looks inside its own folder. A version of that configuration extending it to also reach the appointments folders exists, but it sits on a branch that has not yet merged into this test repo's staging branch. Practically: running the right backend command finds all 57 files; running the wrong one silently reports nothing wrong, because it never looked.

Pre-Release Checklist

Before any sign-off decision, do this:

1. **Redeploy the admin panel.** The UAT admin URL is 96 commits behind, missing 86 appointments-related changes. Any admin-side browser walk performed today is testing stale software — the sign-off is void until this is fixed. *(Cannot be resolved by this document — it requires triggering the actual deploy.)*
2. **Seed at least one bookable service on the `qa-admin` UAT store** (or confirm it is already seeded) before attempting any browser walk of the booking funnel, payment, or reschedule/cancel flows on UAT — no other UAT store currently passes the appointments plan gate.
3. **Run the backend suite with the correct command** (`test:src`, not the delegating `test` script) so the 57 backend appointments specs are actually exercised — quoted as a finding above, not assumed.
4. **Run the admin-panel and customer-portal specs through this project's standard test process** — both are confirmed wired in; no special flag or workaround needed.
5. **Treat the cross-service, end-to-end money flow as manual-only for this release.** An automated version exists but is not merged into the running test suite. Until it merges, a human must walk: create a booking with a deposit → pay the deposit via the local payment stand-in → confirm the invoice and outstanding-balance figures agree → pay the remaining balance → confirm the booking's status and the order's paid-status agree. This cannot be waived by pointing at unit-test coverage alone.
6. **A true "does the SMS/payment-link message actually arrive" check cannot be done in UAT by an automated test** — the notification and payment-link tests confirm the service attempts to send, not that a phone receives it. If this needs proving, it needs a human with a real phone number, once, deliberately.
7. **No automated browser-level (Playwright) walk exists for appointments at all.** Any claim that this module has been "browser-tested" for this release rests entirely on a human doing it by hand — confirm that walk actually happened and ask for its evidence before accepting a sign-off based on it.

Appendix B — What We Are Fixing, and What Turned Out Not to Be Broken

Part 13 lists what isn't finished yet. This appendix says what happens to each of those entries. Every item on that list was re-checked against the product's own code before this release, and the result changed the list in four places: **four things we had listed as limitations were not limitations at all.**

Three of the twenty items are not engineering work and are called out as such — pretending otherwise would just move them from one list to another.

Already correct — Part 13 was wrong about these four

What Part 13 said	What is actually true
Intake **submissions cannot be read**	They can. Every submitted answer shows on that booking's detail panel, formatted correctly for all eleven field types, with a proper empty state. Only the all-in-one roll-up view on the form builder is still a stub — that one is on the fix list, and it is a convenience feature, not missing data.
The customer's **date-and-time screen has no loading skeleton**	It does. Both the service list and the date-and-time screen show a pulsing placeholder while loading — they simply use a different number of bars, matching what each screen is about to show.
Deposit rounding at the moment of calculation was never confirmed	Confirmed correct. Amounts round to the nearest paisa using the safe method, applied at the first calculation and consistently for every deposit type.
The **refund wording** between "initiated" and "landed" was never inspected	Inspected. The customer sees three distinct messages: in-flight (with a 5–7 working day expectation), completed (labelled "Refunded", with the amount), and failed (asking them to contact you).

Part 13 in this edition has been corrected accordingly.

Being fixed first — nothing new to build, only to connect

Each of these already works in the product; it was simply never surfaced.

- **A time limit you can set for unpaid bookings.** The limit already exists and is already enforced on every sweep — it was just missing from the Settings screen. It becomes a field you can change, within a 5-minute to 7-day range.
- **A GST invoice you can download for a booking.** The whole invoice chain already works for bookings: the GST rate and SAC code you set on a service are carried onto the booking's order, and the invoice builder already itemises them. The only thing missing was a link to it from the booking.
- **A warning when a service has nobody assigned.** Today a service with no team member silently offers no times, and the only hint lives inside a resource's Availability tab. You will get the warning where you make the mistake — on the service itself.
- **A "deposit required" badge on the service list.** Your customers currently find out about the deposit at the review step. For any service where you set the deposit explicitly, the badge appears on the card straight away.
- **Plan limits shown before you hit them,** not only after a save is refused. The staff-seat limit already works this way; the rest will match it.

Being fixed next — real work, in priority order

1. **Overlapping working hours will be rejected.** Today two overlapping weekly shifts both save with no warning, which quietly corrupts your availability. This is the highest-value fix on the list.
2. **An expiry on the guest "manage booking" link.** Today that link works for as long as the booking exists — indefinitely, including long after the appointment — and it can cancel or pay for the booking. It will get a time limit, chosen so that a customer re-opening a reminder message weeks later still works.
3. **Spots remaining on group/capacity bookings.** The limit is already enforced correctly; the count just isn't shown to you on the calendar.
4. **Blocking time directly on the calendar,** instead of navigating to a resource's Availability tab.
5. **A message when a booking is rescheduled.** Today the new time is shown on screen and nowhere else — no message type for it exists. This is the one missing message most likely to cause a no-show, so it ships before the others. Check-in, start and completion messages will follow behind a per-event switch, so you are not forced to send them.

6. **Deposit, reminder and message events on the booking's activity timeline.** Reminders are already recorded — they were simply being stripped out before reaching the screen.
7. **Real delivery confirmation for WhatsApp.** "Sent" currently means your provider accepted the message. Delivery and read status will be recorded properly rather than assumed.

Honest about three things that are not fixes

- **SMS payment links stay switched off, and that is correct.** No SMS provider is connected to the platform at all — not for appointments, not for anything else. Switching the option on would promise something nothing could deliver. Turning it on is a commercial decision about a provider, not a code change.
- **Plan-limit safety needs a data check before a code change.** If a plan is missing its row for a feature, access is currently granted rather than refused. Changing that default without first checking every plan's rows would instantly lock out customers who work fine today — across roughly twenty features, not just Appointments. So the plan data gets audited first. Until then, confirming every plan has explicit rows is a per-release check.
- **The reminder scheduler runs as a single instance and that is safe today.** Each reminder is claimed under a database lock before it is sent, so the same reminder cannot go out twice even if the scheduler fired more than once. The broader change is platform infrastructure, unrelated to Appointments.

One thing we are not claiming

Whether guest verification codes are actually enforced before a booking can be confirmed or changed was not re-checked for this release. The code is sent — that part is real — but we did not verify that a booking cannot proceed without it. It is recorded as unmeasured rather than described as working, and it is the first thing being checked. Treat it as an open question during your acceptance pass.

Glossary

Offering — one service a customer can book: a haircut, a consultation, a 90-minute studio session. It carries the price, the duration, the deposit rule and the list of team members who can deliver it. Managed under **Appointments → Offerings**.

Resource — anything that gets reserved for a block of time. A **Person**-type resource is a staff member; other resources are rooms, chairs, bays, or equipment. All of them live under **Team & Resources**.

Staff seat — the plan allowance that lets you have an active Person-type resource. Most plans include zero seats by default; seats are generally bought as an add-on rather than unlocked by a plan tier.

Weekly shift — the recurring working hours of one resource (for example Mon–Fri, 10:00–18:00). A resource with no weekly shift is treated as never available, so it never offers a slot.

Slot — one bookable start time on the grid. Which slots exist on a given day is calculated from the resource's shifts, your slot length, your minimum notice and what's already booked — all in your store's own timezone.

Hold — a temporary 5-minute claim placed on a slot the moment a customer picks it, so the same time isn't offered to someone else while they finish the form. It is advisory only; the real protection against double-booking is enforced in the database. If it lapses, the customer is asked to pick a time again.

Deposit — the amount a customer pays upfront to secure a booking, set per offering as either a fixed amount or a percentage of the price. A deposit greater than zero is what makes a booking start as **Pending payment**.

Deposit timeout — how long an unpaid deposit keeps a booking alive, set in Settings (default 30 minutes). An automatic sweep cancels the booking and frees the slot once it passes. Not the same thing as a hold.

Balance due — what is still owed on a booking after the deposit. The booking's Payment section always shows the live outstanding amount, and the customer's payment screen always asks the server what's owed rather than reusing an earlier figure.

Payment link — a link you can send from a booking's Payment section so the customer can pay what's outstanding themselves.

Record payment — logging a payment you took another way (cash, card machine) against a booking, from the Payment section. The amount is pre-filled with the live outstanding amount.

Booking statuses — **Pending payment** (created, deposit still owed); **Confirmed** (secured — nothing outstanding blocks it); **Checked in** (the customer has arrived); **In progress** (the appointment is happening); **Completed** (finished); **Cancelled** and **No-show** (both final). The customer sees these same labels. A booking with a deposit still owed is never shown to the customer as confirmed.

No-show fee — the percentage you keep when a customer doesn't turn up, set in Settings. What's actually kept can never exceed what was really collected.

Minimum notice — how far in advance a customer must book. It pushes the earliest offered slot forward from now.

Booking horizon — how far into the future customers may book. Dates beyond it show no slots.

Slot length / start increment — how the bookable start times are spaced on the grid.

Cancellation window — the deadline before an appointment after which a customer may no longer cancel or reschedule themselves. Staff and admins have no such restriction and can always cancel.

Intake form — the extra questions a customer answers while booking a specific service (allergies, vehicle registration, and so on). Built per offering, and only after that offering has been saved once. The answers appear on the booking's detail panel.

Capacity (group) service — an offering that takes more than one customer in the same slot, up to a configured limit, as opposed to a one-to-one resource that takes a single booking per slot.

Preset — a business-type starting point in Settings that fills in all your booking rules at once. Applying one overwrites every rule you've already saved, and cannot be undone.

Reminder cadence — the list of times before an appointment at which reminders are sent, entered in Settings as minutes-before. Reminders only go to bookings that are Confirmed or Checked in.

My Day — the simplified personal schedule a staff member sees instead of the full calendar.

My availability — the availability editor as a staff member reaches it, scoped automatically to their own resource.

Guest manage link — the personal link a customer who booked without an account is given, which opens their booking with the same information and the same actions a signed-in customer would have.

Plan gate — the check, run each time an Appointments screen loads, that your subscription includes the module. When it fails you see an upgrade message instead of the calendar.